

50 Most Asked Verilog Coding Interview Questions

A Practical Guide to RTL Coding Interviews Covering Topics Tested At:

NVIDIA • AMD • Intel • Qualcomm • Apple • Google
Synopsys • Cadence • Broadcom • ARM • Samsung • Texas Instruments

Kittu Patel

Beginner to Expert • 50 Real Interview Problems • Production Verilog Code

First Edition • 2025

50 Most Asked Verilog Coding Interview Questions

A Practical Guide to RTL Coding Interviews

Copyright © 2025 by **Kittu K Patel**.

All rights reserved. No part of this publication may be reproduced, distributed, or transmitted in

any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the author, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

Disclaimer: All Verilog code examples in this book are written for educational and interview-

preparation purposes. While every effort has been made to ensure accuracy, the author does not warrant that the code is free from defects or suitable for any particular production application.

Edition: First Edition, 2025

Author: Kittu Patel

Subject: RTL Design, Verilog, FPGA/ASIC Interview Preparation

Language: Verilog / SystemVerilog

Verilog Standards Covered

This book covers **Verilog-2001 (IEEE 1364-2001)** and **SystemVerilog-2012 (IEEE 1800-2012)** constructs. ANSI-style module declarations are used throughout unless stated otherwise. All synthesizable code is written targeting standard ASIC/FPGA flows.

Preface

Why This Book Exists

Verilog and RTL design interviews are among the most technically demanding interviews in the semiconductor industry. Unlike software engineering interviews, where high-level algorithms dominate, RTL interviews require you to *think in hardware*. You must simultaneously reason about:

- The logical behavior of a circuit
- Its physical implementation in gates and flip-flops
- Its timing characteristics under clock constraints
- Its synthesizability and mapping to target technology

Companies such as **NVIDIA**, **AMD**, **Intel**, **Qualcomm**, **Apple**, **Google**, **Broadcom**, **ARM**, **Synopsys**, and **Cadence** consistently report that a significant portion of RTL candidates fail to write correct, synthesizable Verilog under interview conditions — not because they lack knowledge, but because they have never practiced structured, disciplined RTL coding to a professional standard.

How This Book Is Organized

This book contains **50 carefully selected Verilog interview questions**, ordered by difficulty:

Level	Description	Questions	Chapter
Beginner	Fundamental RTL constructs	Q1–Q10	1
Intermediate	Core design blocks	Q11–Q30	2
Advanced	Complex modules & protocols	Q31–Q45	3
Expert	Industry-grade designs	Q46–Q50	4

Each question follows a consistent structure:

1. **Problem Statement** - exactly as posed in interviews
2. **Interview Requirements** - what the interviewer expects
3. **Verilog Solution** - production-quality, synthesizable code
4. **Line-by-Line Explanation** - every construct explained
5. **Common Mistakes** - pitfalls to avoid
6. **Pro Tips** - insider advice from working RTL engineers
7. **Interviewer’s Perspective** - what they are really testing
8. **Follow-Up Questions** - be ready for these

Recommended Study Strategy

How to Use This Book Effectively

1. **Code by hand first.** Before reading the solution, write your own Verilog on paper or a whiteboard. This is exactly what you will do in the interview.
2. **Simulate every example.** Run each solution in ModelSim, VCS, or Icarus Verilog. Seeing waveforms cements understanding.
3. **Study the mistakes section carefully.** Most candidates fail on blocking/non-blocking confusion and latch inference.
4. **Memorize the Pro Tips.** These reflect real industry best practices.
5. **Practice the Follow-Up Questions aloud.** Interview responses must be clear and concise.
6. **Attempt the Practice Challenge.** These exercises build the muscle memory required to code under pressure.

A Note on SystemVerilog

Some solutions include SystemVerilog constructs (e.g., `always_ff`, `always_comb`, `logic`). When a SystemVerilog construct appears, its equivalent Verilog-2001 version is also shown, because many companies still mandate pure Verilog in whiteboard interviews.

Good luck with your interviews. Remember: *hardware is designed, not coded*. The best RTL engineers think first in circuits, then translate that thinking into Verilog. This book teaches you to do exactly that.

— **Kittu Patel**

Contents

Preface	iii
1 Beginner Questions	1
Q1: Design a 4-to-1 Multiplexer	2
Q2: Design a 3-to-8 Decoder	4
Q3: Design an 8-bit Priority Encoder	6
Q4: Design a Parameterized D Flip-Flop with Synchronous Reset	8
Q5: Design a 4-bit Synchronous Up Counter	10
Q6: Design a Universal Shift Register	12
Q7: Design a Parameterized Comparator	14
Q8: Design a Moore FSM — Sequence Detector (101)	16
Q9: Design a Parity Generator and Checker	18
Q10: Design a Clock Divider (Divide-by-N)	20
2 Beginner Questions	23
Q1: Design a 4-to-1 Multiplexer	24
Q2: Design a 3-to-8 Decoder	26
Q3: Design an 8-bit Priority Encoder	28
Q4: Design a Parameterized D Flip-Flop with Synchronous Reset	30
Q5: Design a 4-bit Synchronous Up Counter	32
Q6: Design a Universal Shift Register	34
Q7: Design a Parameterized Comparator	36
Q8: Design a Moore FSM — Sequence Detector (101)	38
Q9: Design a Parity Generator and Checker	40
Q10: Design a Clock Divider (Divide-by-N)	42
3 Intermediate Questions	45
Q11: Design a Parameterized Synchronous FIFO	46
Q12: Design a Barrel Shifter	49
Q13: Design a Simple ALU	51
Q14: Design a Mealy FSM — Traffic Light Controller	53
Q15: Design a Single-Port Synchronous RAM	55
Q16: Design a Register File	57
Q17: Design a UART Transmitter	59
Q18: Design an SPI Master	61
Q19: Design a Two-Flop CDC Synchronizer	64
Q20: Design a Round-Robin Arbiter	66
Q21: Design a PWM Generator	68
Q22: Design a Johnson Counter	69
Q23: Design a Gray Code Counter	70
Q24: Design a CRC-8 Generator	72
Q25: Design a Parameterized Pipeline Register Stage	74
Q26: Design a Vending Machine FSM	76
Q27: Design a Debouncer Circuit	78
Q28: Design an Up/Down Counter with Load	80
Q29: Design a Pulse Stretcher / Monostable Multivibrator	81
Q30: Design a Positive and Negative Edge Detector	83

4 Expert Questions	85
Q46: Design a Parameterized Carry-Lookahead Adder	86
Q47: Design a Two-Stage Out-of-Order Issue Queue	89
Q48: Design a Low-Power Handshake Protocol (4-Phase)	92
Q49: Design a Full AXI4 Lite Slave with Register Map	95
Q50: Design a Non-Blocking Cache with MSHR	98
A Verilog Quick Reference	103
A.1 Blocking vs Non-Blocking Assignment	103
A.2 Sensitivity List Rules	103
A.3 Latch Inference — How to Avoid	104
A.4 Key Verilog Operators	104
A.5 Common Design Patterns	104
A.6 FIFO Pointer Full/Empty Rules	105
A.7 FSM Design Rules	105
A.8 Clock Domain Crossing Cheat Sheet	105
B Interview Preparation Strategy	107
B.1 30-Day Study Plan	107
B.2 On-Site Interview Tips	107
B.3 Topics by Company Focus	107
C Recommended Reading	109

Beginner Questions

Chapter Overview

These ten questions test fundamental RTL concepts. Every RTL interview starts here. Master these before moving to later chapters. Topics: Combinational logic, basic sequential circuits, encoders, decoders, muxes, counters, shift registers, and basic FSMs.

Question 1

Design a 4-to-1 Multiplexer

BEGINNER

Companies That Ask This Question

NVIDIA, Qualcomm, Intel, Texas Instruments, ARM, MediaTek

Problem Statement

Design a 4-to-1 multiplexer with a 2-bit select input. When `sel = 2'b00` output `y = in0`, `sel = 2'b01` gives `y = in1`, and so on. Write a parameterized version supporting configurable data width.

Interview Requirements: The interviewer wants to see clean `case` or `always` style, parameter usage, and correct combinational inference. **Constraints:** Fully combinational. No latches. Data width parameterized.

Verilog Solution - mux4to1.v

```

1      // -----
2      // Module   : mux4to1
3      // Purpose  : Parameterized 4-to-1 Multiplexer
4      // Author   : Kittu Patel
5      // -----
6      module mux4to1 #(
7          parameter DATA_WIDTH = 8          // Configurable data width
8      )(
9          input  wire [DATA_WIDTH-1:0] in0, // Input channel 0
10         input  wire [DATA_WIDTH-1:0] in1, // Input channel 1
11         input  wire [DATA_WIDTH-1:0] in2, // Input channel 2
12         input  wire [DATA_WIDTH-1:0] in3, // Input channel 3
13         input  wire [1:0] sel,           // 2-bit select
14         output reg  [DATA_WIDTH-1:0] y   // Selected output
15     );
16
17     always @(*) begin
18         case (sel)
19             2'b00: y = in0;
20             2'b01: y = in1;
21             2'b10: y = in2;
22             2'b11: y = in3;
23             default: y = {DATA_WIDTH{1'b0}}; // Avoid latch
24         endcase
25     end
26 endmodule
27

```

Line-by-Line Explanation:

- `parameter DATA_WIDTH = 8` — makes the module reusable for any bus width.
- `always @(*)` — synthesizes to pure combinational logic (no registers).
- `case (sel)` — maps directly to a 4:1 MUX gate structure.
- `default: y = 0` — prevents latch inference when `sel` has metastable value in simulation.

RTL Design Explanation: A 4:1 MUX is purely combinational. The synthesizer implements it with three 2:1 MUX cells. The `default` branch ensures a complete case table, preventing unintended latch creation.

Common Interview Mistakes

- **Missing `default`:** Omitting `default` in `case` causes a latch on `y` because `sel = 2'bxx` has no assignment.
- **Using `reg` in `assign`:** Writing `assign y = ...` and declaring `y` as `reg` is a type mismatch.
- **Forgetting parameters:** Hard-coding 8-bit width is unprofessional. Interviewers always ask "Can you make it parameterized?" next.

Pro Tips

- Use `always_comb` in SystemVerilog — it automatically adds all RHS signals to the sensitivity list and flags unintended latches at compile time.
- For very wide MUXes, `generate` + `for` loops create efficient structures that synthesize well.
- In FPGA targets, MUX is a native LUT primitive — no extra cells needed.

Interviewer's Perspective

Why asked: Tests basic understanding of combinational logic coding styles.

Evaluating: Correct sensitivity list, case completeness, parameterization.

Follow-ups: "Make it 8:1", "Convert to structural Verilog", "What if inputs are 32-bit?"

Impress: Immediately propose parameterization without being asked.

Follow-Up Questions

1. Design an 8-to-1 MUX using two 4-to-1 MUXes.
2. Implement a MUX using only 2-input NAND gates.
3. How does a MUX synthesize on an FPGA (LUT structure)?
4. What is the critical path delay of a 32:1 MUX tree?
5. Convert this to a priority MUX — what changes?

Practice Challenge

Design a parameterized 2^N -to-1 MUX using a `generate` loop. The parameter `N` controls the number of select bits. Test with `N=2,3,4`.

Question 2

Design a 3-to-8 Decoder

BEGINNER

Companies That Ask This Question

Intel, Synopsys, Cadence, Samsung, Marvell

Problem Statement

Design a 3-to-8 binary decoder with an active-high enable input. When `en = 1`, exactly one of the 8 output lines is asserted high based on the 3-bit input `in`. When `en = 0`, all outputs are low.

Verilog Solution - decoder3to8.v

```

1      // -----
2      // Module : decoder3to8
3      // Purpose : 3-to-8 Binary Decoder with Enable
4      // -----
5      module decoder3to8 (
6          input wire [2:0] in,    // 3-bit binary input
7          input wire en,         // Active-high enable
8          output reg [7:0] out    // One-hot decoded output
9      );
10     always @(*) begin
11         if (!en) begin
12             out = 8'h00;        // All outputs low when disabled
13         end else begin
14             out = 8'h00;        // Default: clear all
15             out[in] = 1'b1;     // Assert the selected line
16         end
17     end
18 endmodule
19

```

RTL Design Explanation: `out[in] = 1'b1` is an elegant variable bit-select. The synthesizer maps this to 8 AND gates, each gating `en` with the decoded min-term. The default assignment prevents latches.

Common Interview Mistakes

- Writing a 9-row case statement instead of using the elegant `out[in]` idiom.
- Forgetting `out = 8'h00` before the enable check — causes latch.
- Using `assign out = en ? (8'h01 << in) : 8'h00` — valid but harder to read in interviews.

Pro Tips

The expression `(8'h01 << in)` is the shift-based alternative and is highly synthesizable. Both styles are acceptable; know both for the interview.

Interviewer's Perspective

Why asked: Decoders are critical in address decoding (memory maps, register files).

Evaluating: Combinational coding style, enable logic, completeness.

Follow-ups: “Add active-low enable”, “Use this decoder in a RAM address decode”.

Follow-Up Questions

1. Implement a 4-to-16 decoder using two 3-to-8 decoders.
2. What is the output when `en=0` in a typical SRAM address decoder?
3. How does a decoder differ from a demultiplexer?
4. Design a BCD-to-7-segment decoder.
5. What happens if two enable signals overlap? (glitch analysis)

Practice Challenge

Design a BCD-to-7-segment display decoder. Each segment (a–g) should be correctly driven for digits 0–9. Output `7'bxxxxxxx` for invalid BCD inputs (10–15).

Question 3

Design an 8-bit Priority Encoder

BEGINNER

Companies That Ask This Question

Qualcomm, ARM, NVIDIA, Broadcom, AMD

Problem Statement

Design an 8-input priority encoder. The output `out` is the binary encoding of the highest-priority (MSB-first) asserted input. Include a `valid` output that is high when at least one input is asserted.

Verilog Solution - priority_enc8.v

```

1      // -----
2      // Module : priority_enc8
3      // Purpose : 8-input Priority Encoder (MSB = highest priority)
4      // -----
5      module priority_enc8 (
6          input wire [7:0] in,      // 8 request lines
7          output reg [2:0] out,     // 3-bit encoded output
8          output reg      valid    // At least one input active
9      );
10     always @(*) begin
11         // Default: no valid request
12         out  = 3'b000;
13         valid = 1'b0;
14         // Priority: in[7] is highest, in[0] is lowest
15         if (in[7]) begin out = 3'd7; valid = 1'b1; end
16         else if (in[6]) begin out = 3'd6; valid = 1'b1; end
17         else if (in[5]) begin out = 3'd5; valid = 1'b1; end
18         else if (in[4]) begin out = 3'd4; valid = 1'b1; end
19         else if (in[3]) begin out = 3'd3; valid = 1'b1; end
20         else if (in[2]) begin out = 3'd2; valid = 1'b1; end
21         else if (in[1]) begin out = 3'd1; valid = 1'b1; end
22         else if (in[0]) begin out = 3'd0; valid = 1'b1; end
23     end
24     endmodule
25

```

RTL Design Explanation: The `if-else if` chain creates a priority structure. The synthesizer generates a network of comparators / mux trees that evaluates from MSB to LSB. The `valid` flag is a simple OR of all inputs.

Common Interview Mistakes

- Using `case` instead of `if-else if` — `casez/casex` can work but the priority semantics must be explicitly verified.
- Forgetting the `valid` signal entirely.
- Swapping MSB/LSB priority direction.

Pro Tips

In industry, priority encoders appear in interrupt controllers, arbiters, and out-of-order CPUs. A **find-first-set** (FFS) instruction is essentially a hardware priority encoder. Know how to build parameterized log-depth trees for large encoders ($N > 32$).

Follow-Up Questions

1. Design a 16-input priority encoder using two 8-input encoders.
2. What is the gate-level structure of a priority encoder?
3. How does a priority encoder differ from a regular encoder?
4. Design a priority encoder using `casez`.
5. Where is a priority encoder used in an interrupt controller?

Question 4

Design a Parameterized D Flip-Flop with Synchronous Reset

BEGINNER

Companies That Ask This Question

Intel, AMD, Synopsys, Cadence, Texas Instruments, Apple

Problem Statement

Design a parameterized D flip-flop with synchronous active-high reset. When `rst = 1` on the rising clock edge, the output resets to 0. Include a synchronous load enable (`en`).

Verilog Solution - dff_sync.v

```

1      // -----
2      // Module   : dff_sync
3      // Purpose  : Parameterized D-FF, Synchronous Reset + Enable
4      // -----
5      module dff_sync #(
6          parameter WIDTH = 8,
7          parameter RESET_VAL = 0
8      )(
9          input wire      clk,
10         input wire      rst,    // Synchronous, active-high
11         input wire      en,     // Load enable
12         input wire [WIDTH-1:0] d,
13         output reg [WIDTH-1:0] q
14     );
15     always @(posedge clk) begin
16         if (rst)      q <= {WIDTH{1'b0}};    // Sync reset
17         else if (en)  q <= d;                // Load when enabled
18         // else: hold current value (implicit)
19     end
20 endmodule
21

```

Timing Discussion: The critical path runs from `d` through the MUX selecting between `d` and `q` to the D input of the flip-flop. Reset has the same path timing as data. Setup time must hold for all three paths: `rst`, `en`, and `d`.

Common Interview Mistakes

- **Blocking assignment in sequential logic:** Writing `q = d` inside `always @(posedge clk)` instead of `q <= d`. This is the most common RTL interview error worldwide.
- **Async vs sync reset confusion:** Adding `negedge rst` to the sensitivity list converts to asynchronous reset — a completely different circuit.
- **Width mismatch:** Resetting with `q <= 0` (integer) instead of `q <= {WIDTH{1'b0}}` causes simulator warnings.

Pro Tips

- **Sync reset** is preferred in ASIC flows — it avoids clock domain issues and the reset net is not on the critical path.
- **Async reset** is common in FPGA flows because FFs have dedicated async set/reset pins. Know when to use each.
- Always use `<=` (non-blocking) for all flip-flop assignments. This models the actual behavior of physical D flip-flops.

Interviewer's Perspective

Why asked: The flip-flop is the atom of all sequential logic.

Evaluating: Blocking vs non-blocking, sensitivity list correctness, reset style awareness.

Follow-ups: “Async reset version”, “Async reset with sync de-assert”, “Add preset”, “What if WIDTH=1?”

Follow-Up Questions

1. Redesign with asynchronous active-low reset.
2. Design an async reset, synchronous de-assert flip-flop.
3. What is the setup and hold time window?
4. How does the synthesizer implement the enable (`en`) signal?
5. What is clock gating, and how does it relate to flip-flop enable?

Question 5

Design a 4-bit Synchronous Up Counter

BEGINNER

Companies That Ask This Question

Texas Instruments, Intel, Qualcomm, Samsung, MediaTek

Problem Statement

Design a parameterized synchronous binary up counter with synchronous reset, load enable, and a terminal count output (`tc`) that pulses for one clock cycle when the counter reaches its maximum value.

Verilog Solution - counter_up.v

```

1      // -----
2      // Module : counter_up
3      // Purpose : Parameterized Synchronous Up Counter
4      // -----
5      module counter_up #(
6          parameter WIDTH = 4
7      )(
8          input wire      clk,
9          input wire      rst,    // Synchronous reset
10         input wire      en,     // Count enable
11         output reg [WIDTH-1:0] count,
12         output wire      tc     // Terminal count
13     );
14     localparam MAX_VAL = (1 << WIDTH) - 1;
15     // Terminal count: asserted when counter is at maximum
16     assign tc = (count == MAX_VAL[WIDTH-1:0]) & en;
17     always @(posedge clk) begin
18         if (rst)      count <= {WIDTH{1'b0}};
19         else if (en)  count <= count + 1'b1;
20     end
21 endmodule
22

```

Hardware Interpretation: A WIDTH-bit counter synthesizes to WIDTH flip-flops and a ripple-carry adder. The terminal count comparator is a XNOR tree (all bits must equal 1) AND-gated with `en`. **Timing Discussion:** The critical path is the carry propagation through the adder. For a 32-bit counter, this is 32 XOR/AND delays. A carry-lookahead adder reduces this to $O(\log N)$.

Common Interview Mistakes

- Defining `tc` inside `always` block as `reg` and adding a one-cycle delay to terminal count.
- Using `count + 1` (32-bit integer addition) causing width expansion warnings. Always use `count + 1'b1` or `count + {{(WIDTH-1){1'b0}}, 1'b1}`.
- Forgetting that `localparam MAX_VAL` is an integer by default — always slice or cast it to the correct width.

Follow-Up Questions

1. Add a synchronous load input (`load`, `load_val`).
2. Convert to a down counter with borrow output.
3. Design a modulo-N counter (counts 0 to N-1).
4. How do you implement a BCD counter (0 to 9)?
5. What is the maximum clock frequency limited by?

Practice Challenge

Design a modulo-N counter where N is a parameter. The counter should wrap from N-1 back to 0. Include both `tc` (terminal count) and a configurable initial value `INIT`.

Question 6

Design a Universal Shift Register

BEGINNER

Companies That Ask This Question

Intel, NVIDIA, AMD, Broadcom, ARM

Problem Statement

Design a parameterized universal shift register that supports four modes via a 2-bit control: 00=Hold, 01=Shift-Right, 10=Shift-Left, 11=Parallel Load.

Verilog Solution - shift_reg_universal.v

```

1      //-----
2      // Module : shift_reg_universal
3      // Purpose : Universal Shift Register (4 modes)
4      //-----
5      module shift_reg_universal #(
6          parameter WIDTH = 8
7      )(
8          input wire      clk,
9          input wire      rst,
10         input wire [1:0] ctrl,      // Mode select
11         input wire [WIDTH-1:0] d_parallel, // Parallel data in
12         input wire      d_serial_r, // Serial in (right)
13         input wire      d_serial_l, // Serial in (left)
14         output reg [WIDTH-1:0] q,
15         output wire      q_serial_r, // LSB serial out
16         output wire      q_serial_l // MSB serial out
17     );
18     localparam HOLD      = 2'b00;
19     localparam SHIFT_RIGHT = 2'b01;
20     localparam SHIFT_LEFT  = 2'b10;
21     localparam PARALLEL_LD = 2'b11;
22     assign q_serial_r = q[0];      // Serial out for right shift
23     assign q_serial_l = q[WIDTH-1]; // Serial out for left shift
24     always @(posedge clk) begin
25         if (rst) begin
26             q <= {WIDTH{1'b0}};
27         end else begin
28             case (ctrl)
29                 HOLD:      q <= q;          // Hold
30                 SHIFT_RIGHT: q <= {d_serial_r, q[WIDTH-1:1]}; // Shift right
31                 SHIFT_LEFT:  q <= {q[WIDTH-2:0], d_serial_l}; // Shift left
32                 PARALLEL_LD: q <= d_parallel; // Parallel load
33             endcase
34         end
35     end
36 endmodule
37

```

Common Interview Mistakes

- Shift direction confusion: Right shift inserts from MSB (`{d_serial_r, q[N-1:1]}`), not LSB. Sketch the bit positions before coding.
- Forgetting to expose serial outputs — interviewers always ask for these.
- Omitting the `HOLD` case without a `default` — creates a latch.

Follow-Up Questions

1. Add a rotate-right and rotate-left mode (ctrl becomes 3-bit).
2. Use this as a PISO (parallel-in, serial-out) converter.
3. Implement a LFSR (Linear Feedback Shift Register) using this module.
4. How does a barrel shifter differ from this register?
5. Design a ring counter using the shift-right mode.

Question 7

Design a Parameterized Comparator

BEGINNER

Companies That Ask This Question

Intel, Texas Instruments, Samsung, Marvell, Synopsys

Problem Statement

Design a parameterized magnitude comparator. Given inputs **a** and **b**, output three flags: **eq** ($a==b$), **gt** ($a>b$), **lt** ($a<b$). Support both unsigned and signed comparison via a parameter.

Verilog Solution - comparator.v

```

1      // -----
2      // Module : comparator
3      // Purpose : Parameterized Magnitude Comparator
4      // -----
5      module comparator #(
6          parameter WIDTH = 8,
7          parameter SIGNED = 0    // 1 = signed, 0 = unsigned
8      )(
9          input wire [WIDTH-1:0] a,
10         input wire [WIDTH-1:0] b,
11         output wire          eq,    // a == b
12         output wire          gt,    // a > b
13         output wire          lt     // a < b
14     );
15     generate
16     if (SIGNED) begin : signed_cmp
17         wire signed [WIDTH-1:0] sa = a;
18         wire signed [WIDTH-1:0] sb = b;
19         assign eq = (sa == sb);
20         assign gt = (sa > sb);
21         assign lt = (sa < sb);
22     end else begin : unsigned_cmp
23         assign eq = (a == b);
24         assign gt = (a > b);
25         assign lt = (a < b);
26     end
27     endgenerate
28     endmodule
29 
```

Pro Tips

The **generate** block is the right way to handle *compile-time* parameter-driven structural differences. The synthesizer will completely eliminate the unused branch. Signed comparison on unsigned **reg/wire** is achieved with the **\$signed()** cast.

Follow-Up Questions

1. How does signed comparison differ in two's complement arithmetic?
2. Design a comparator using only subtraction.
3. What is the gate-level implementation for `eq`?
4. Build a 32-bit comparator from two 16-bit comparators.
5. When does `gt = !lt & !eq` hold, and when does it not?

Question 8

Design a Moore FSM — Sequence Detector (101)

BEGINNER

Companies That Ask This Question

Intel, Qualcomm, NVIDIA, AMD, Texas Instruments, ARM

Problem Statement

Design a Moore-style FSM that detects the serial bit sequence **101**. The output **detected** is asserted for one cycle after the complete sequence is observed. Support overlapping detection.

Verilog Solution - seq_detect_101.v

```

1      // -----
2      // Module : seq_detect_101 (Moore FSM)
3      // Purpose : Detects overlapping sequence "101"
4      // -----
5      module seq_detect_101 (
6          input wire clk,
7          input wire rst,
8          input wire din,          // Serial bit input
9          output reg detected      // Pulse when "101" detected
10     );
11     // State encoding
12     typedef enum reg [1:0] {
13         S0 = 2'b00, // Initial / reset state
14         S1 = 2'b01, // Got "1"
15         S2 = 2'b10, // Got "10"
16     } state_t;
17     state_t curr_state, next_state;
18     // State register
19     always @(posedge clk) begin
20         if (rst) curr_state <= S0;
21         else curr_state <= next_state;
22     end
23     // Next state logic
24     always @(*) begin
25         case (curr_state)
26             S0: next_state = din ? S1 : S0;
27             S1: next_state = din ? S1 : S2;
28             S2: next_state = din ? S3 : S0;
29             S3: next_state = din ? S1 : S2; // Overlap: "1" from "101" counts
30             default: next_state = S0;
31         endcase
32     end
33     // Output logic (Moore: depends only on current state)
34     always @(*) begin
35         detected = (curr_state == S3);
36     end
37 endmodule
38
39

```

RTL Design Explanation: Three separate always blocks implement the canonical 3-block FSM style: (1) state register, (2) next-state logic, (3) output logic. This is the industry-standard

approach for FSMs.

Common Interview Mistakes

- **2-block vs 3-block FSM:** Merging next-state and output in one block works but is harder to review and synthesize cleanly.
- **Moore vs Mealy confusion:** Moore output depends only on state. Mealy output also depends on the input — mention this distinction.
- **No overlap:** Going back to S0 after detection misses the overlapping case. The interviewer will test the edge case “101101”.

Interviewer’s Perspective

Why asked: FSMs are the backbone of controllers, protocol decoders, and parsers.

Evaluating: State diagram correctness, coding style, Moore/Mealy distinction, reset handling, overlap handling.

Impress: Draw the state diagram first, then write code. Interviewers love this.

Follow-Up Questions

1. Convert this to a Mealy FSM. How does the output timing differ?
2. Detect the sequence “1011” with overlapping.
3. What happens if `rst` is asynchronous?
4. How would you verify this FSM with a testbench?
5. Implement this FSM with one-hot encoding.

Question 9

Design a Parity Generator and Checker

BEGINNER

Companies That Ask This Question

Intel, Texas Instruments, Synopsys, Marvell, Samsung

Problem Statement

Design a parameterized even/odd parity generator. Also design a parity checker that asserts an **error** flag when received data parity does not match the expected parity bit.

Verilog Solution - parity.v

```

1      // -----
2      // Module : parity_gen
3      // Purpose : Parameterized Even/Odd Parity Generator
4      // -----
5      module parity_gen #(
6          parameter WIDTH      = 8,
7          parameter ODD_PARITY = 0 // 0=even, 1=odd
8      )(
9          input  wire [WIDTH-1:0] data,
10         output wire          parity_out
11     );
12     wire even_parity;
13     assign even_parity = ^data; // XOR reduction = even parity bit
14     assign parity_out  = ODD_PARITY ? ~even_parity : even_parity;
15     endmodule
16     // -----
17     // Module : parity_chk
18     // Purpose : Parity Checker
19     // -----
20     module parity_chk #(
21         parameter WIDTH      = 8,
22         parameter ODD_PARITY = 0
23     )(
24         input  wire [WIDTH-1:0] data,
25         input  wire          parity_in, // Received parity bit
26         output wire          error     // High if parity mismatch
27     );
28     wire expected;
29     wire even_p = ^data;
30     assign expected = ODD_PARITY ? ~even_p : even_p;
31     assign error    = (parity_in != expected);
32     endmodule
33

```

Pro Tips

The XOR reduction operator `^data` is one of Verilog's most elegant constructs. It computes the XOR of all bits in one statement. In hardware, this is a tree of XOR gates with $O(\log N)$ depth.

Follow-Up Questions

1. What parity errors can parity *not* detect?
2. How many error-detecting bits are needed for a Hamming code?
3. Implement a 2D parity check (row and column parity).
4. How does parity differ from CRC?
5. Design a SECDED (Single Error Correct, Double Error Detect) ECC.

Question 10

Design a Clock Divider (Divide-by-N)

BEGINNER

Companies That Ask This Question

NVIDIA, AMD, Intel, Qualcomm, Apple, Broadcom

Problem Statement

Design a parameterized clock divider that produces an output clock `clk_out` at $1/N$ of the input clock frequency. For even N , the output should have 50% duty cycle. The module should be synthesizable.

Verilog Solution - clk_divider.v

```

1      // -----
2      // Module   : clk_divider
3      // Purpose  : Parameterized Clock Divider (even N, 50% duty)
4      // Note     : For use as functional reference; in production,
5      //           use dedicated clock buffers / PLL primitives.
6      // -----
7      module clk_divider #(
8          parameter DIV = 4    // Must be even and >= 2
9      )(
10         input  wire clk,
11         input  wire rst,
12         output reg  clk_out
13     );
14     localparam HALF = DIV / 2;
15     reg [$clog2(DIV):0] count; // Counter wide enough for DIV
16     always @(posedge clk) begin
17         if (rst) begin
18             count <= 0;
19             clk_out <= 1'b0;
20         end else begin
21             if (count == HALF - 1) begin
22                 clk_out <= ~clk_out; // Toggle every HALF cycles
23                 count <= 0;
24             end else begin
25                 count <= count + 1'b1;
26             end
27         end
28     end
29 endmodule
30

```

Hardware Interpretation: This synthesizes to a counter plus a T-flip-flop (the toggle of `clk_out`). The counter is the modulo-HALF counter with a terminal compare.

Important Note for Interviews

Interviewers will ask: "Can this clock drive flip-flops?" The correct answer is: **Only with a clock buffer**. In ASIC flows, derived clocks must go through a clock tree. In FPGA flows, use BUFG primitives. Never drive flip-flop clocks directly from `reg` outputs in production

designs.

Common Interview Mistakes

- Generating a non-50% duty cycle clock (counting to N, toggling once).
- Not using `$clog2` — hard-coded counter width limits parameterization.
- Forgetting that derived clocks need clock buffers in real designs.

Follow-Up Questions

1. How do you divide by an odd number and maintain 50% duty cycle?
2. What is the difference between a clock divider and a clock gating cell?
3. How does a PLL generate divided/multiplied clocks?
4. What CDC issues arise when crossing from `clk` to `clk_out` domains?
5. Implement divide-by-3 with 50% duty cycle (requires both edges).

Beginner Questions

Chapter Overview

These ten questions test fundamental RTL concepts. Every RTL interview starts here. Master these before moving to later chapters. Topics: Combinational logic, basic sequential circuits, encoders, decoders, muxes, counters, shift registers, and basic FSMs.

Question 1

Design a 4-to-1 Multiplexer

BEGINNER

Companies That Ask This Question

NVIDIA, Qualcomm, Intel, Texas Instruments, ARM, MediaTek

Problem Statement

Design a 4-to-1 multiplexer with a 2-bit select input. When `sel = 2'b00` output `y = in0`, `sel = 2'b01` gives `y = in1`, and so on. Write a parameterized version supporting configurable data width.

Interview Requirements: The interviewer wants to see clean `case` or `always` style, parameter usage, and correct combinational inference. **Constraints:** Fully combinational. No latches. Data width parameterized.

Verilog Solution - mux4to1.v

```

1      // -----
2      // Module   : mux4to1
3      // Purpose  : Parameterized 4-to-1 Multiplexer
4      // Author   : Kittu Patel
5      // -----
6      module mux4to1 #(
7          parameter DATA_WIDTH = 8          // Configurable data width
8      )(
9          input  wire [DATA_WIDTH-1:0] in0, // Input channel 0
10         input  wire [DATA_WIDTH-1:0] in1, // Input channel 1
11         input  wire [DATA_WIDTH-1:0] in2, // Input channel 2
12         input  wire [DATA_WIDTH-1:0] in3, // Input channel 3
13         input  wire [1:0] sel,           // 2-bit select
14         output reg  [DATA_WIDTH-1:0] y   // Selected output
15     );
16
17     always @(*) begin
18         case (sel)
19             2'b00: y = in0;
20             2'b01: y = in1;
21             2'b10: y = in2;
22             2'b11: y = in3;
23             default: y = {DATA_WIDTH{1'b0}}; // Avoid latch
24         endcase
25     end
26 endmodule
27

```

Line-by-Line Explanation:

- `parameter DATA_WIDTH = 8` — makes the module reusable for any bus width.
- `always @(*)` — synthesizes to pure combinational logic (no registers).
- `case (sel)` — maps directly to a 4:1 MUX gate structure.
- `default: y = 0` — prevents latch inference when `sel` has metastable value in simulation.

RTL Design Explanation: A 4:1 MUX is purely combinational. The synthesizer implements it with three 2:1 MUX cells. The `default` branch ensures a complete case table, preventing unintended latch creation.

Common Interview Mistakes

- **Missing `default`:** Omitting `default` in `case` causes a latch on `y` because `sel = 2'bxx` has no assignment.
- **Using `reg` in `assign`:** Writing `assign y = ...` and declaring `y` as `reg` is a type mismatch.
- **Forgetting parameters:** Hard-coding 8-bit width is unprofessional. Interviewers always ask "Can you make it parameterized?" next.

Pro Tips

- Use `always_comb` in SystemVerilog — it automatically adds all RHS signals to the sensitivity list and flags unintended latches at compile time.
- For very wide MUXes, `generate` + `for` loops create efficient structures that synthesize well.
- In FPGA targets, MUX is a native LUT primitive — no extra cells needed.

Interviewer's Perspective

Why asked: Tests basic understanding of combinational logic coding styles.

Evaluating: Correct sensitivity list, case completeness, parameterization.

Follow-ups: "Make it 8:1", "Convert to structural Verilog", "What if inputs are 32-bit?"

Impress: Immediately propose parameterization without being asked.

Follow-Up Questions

1. Design an 8-to-1 MUX using two 4-to-1 MUXes.
2. Implement a MUX using only 2-input NAND gates.
3. How does a MUX synthesize on an FPGA (LUT structure)?
4. What is the critical path delay of a 32:1 MUX tree?
5. Convert this to a priority MUX — what changes?

Practice Challenge

Design a parameterized 2^N -to-1 MUX using a `generate` loop. The parameter `N` controls the number of select bits. Test with `N=2,3,4`.

Question 2

Design a 3-to-8 Decoder

BEGINNER

Companies That Ask This Question

Intel, Synopsys, Cadence, Samsung, Marvell

Problem Statement

Design a 3-to-8 binary decoder with an active-high enable input. When `en = 1`, exactly one of the 8 output lines is asserted high based on the 3-bit input `in`. When `en = 0`, all outputs are low.

Verilog Solution - decoder3to8.v

```

1      // -----
2      // Module : decoder3to8
3      // Purpose : 3-to-8 Binary Decoder with Enable
4      // -----
5      module decoder3to8 (
6          input wire [2:0] in,    // 3-bit binary input
7          input wire en,         // Active-high enable
8          output reg [7:0] out    // One-hot decoded output
9      );
10     always @(*) begin
11         if (!en) begin
12             out = 8'h00;        // All outputs low when disabled
13         end else begin
14             out = 8'h00;        // Default: clear all
15             out[in] = 1'b1;     // Assert the selected line
16         end
17     end
18 endmodule
19

```

RTL Design Explanation: `out[in] = 1'b1` is an elegant variable bit-select. The synthesizer maps this to 8 AND gates, each gating `en` with the decoded min-term. The default assignment prevents latches.

Common Interview Mistakes

- Writing a 9-row case statement instead of using the elegant `out[in]` idiom.
- Forgetting `out = 8'h00` before the enable check — causes latch.
- Using `assign out = en ? (8'h01 << in) : 8'h00` — valid but harder to read in interviews.

Pro Tips

The expression `(8'h01 << in)` is the shift-based alternative and is highly synthesizable. Both styles are acceptable; know both for the interview.

Interviewer's Perspective

Why asked: Decoders are critical in address decoding (memory maps, register files).

Evaluating: Combinational coding style, enable logic, completeness.

Follow-ups: “Add active-low enable”, “Use this decoder in a RAM address decode”.

Follow-Up Questions

1. Implement a 4-to-16 decoder using two 3-to-8 decoders.
2. What is the output when `en=0` in a typical SRAM address decoder?
3. How does a decoder differ from a demultiplexer?
4. Design a BCD-to-7-segment decoder.
5. What happens if two enable signals overlap? (glitch analysis)

Practice Challenge

Design a BCD-to-7-segment display decoder. Each segment (a–g) should be correctly driven for digits 0–9. Output `7'bxxxxxxx` for invalid BCD inputs (10–15).

Question 3

Design an 8-bit Priority Encoder

BEGINNER

Companies That Ask This Question

Qualcomm, ARM, NVIDIA, Broadcom, AMD

Problem Statement

Design an 8-input priority encoder. The output `out` is the binary encoding of the highest-priority (MSB-first) asserted input. Include a `valid` output that is high when at least one input is asserted.

Verilog Solution - priority_enc8.v

```

1      // -----
2      // Module : priority_enc8
3      // Purpose : 8-input Priority Encoder (MSB = highest priority)
4      // -----
5      module priority_enc8 (
6          input wire [7:0] in,      // 8 request lines
7          output reg [2:0] out,     // 3-bit encoded output
8          output reg      valid    // At least one input active
9      );
10     always @(*) begin
11         // Default: no valid request
12         out  = 3'b000;
13         valid = 1'b0;
14         // Priority: in[7] is highest, in[0] is lowest
15         if (in[7]) begin out = 3'd7; valid = 1'b1; end
16         else if (in[6]) begin out = 3'd6; valid = 1'b1; end
17         else if (in[5]) begin out = 3'd5; valid = 1'b1; end
18         else if (in[4]) begin out = 3'd4; valid = 1'b1; end
19         else if (in[3]) begin out = 3'd3; valid = 1'b1; end
20         else if (in[2]) begin out = 3'd2; valid = 1'b1; end
21         else if (in[1]) begin out = 3'd1; valid = 1'b1; end
22         else if (in[0]) begin out = 3'd0; valid = 1'b1; end
23     end
24     endmodule
25

```

RTL Design Explanation: The `if-else if` chain creates a priority structure. The synthesizer generates a network of comparators / mux trees that evaluates from MSB to LSB. The `valid` flag is a simple OR of all inputs.

Common Interview Mistakes

- Using `case` instead of `if-else if` — `casez/casex` can work but the priority semantics must be explicitly verified.
- Forgetting the `valid` signal entirely.
- Swapping MSB/LSB priority direction.

Pro Tips

In industry, priority encoders appear in interrupt controllers, arbiters, and out-of-order CPUs. A **find-first-set** (FFS) instruction is essentially a hardware priority encoder. Know how to build parameterized log-depth trees for large encoders ($N > 32$).

Follow-Up Questions

1. Design a 16-input priority encoder using two 8-input encoders.
2. What is the gate-level structure of a priority encoder?
3. How does a priority encoder differ from a regular encoder?
4. Design a priority encoder using `casez`.
5. Where is a priority encoder used in an interrupt controller?

Question 4

Design a Parameterized D Flip-Flop with Synchronous Reset

BEGINNER

Companies That Ask This Question

Intel, AMD, Synopsys, Cadence, Texas Instruments, Apple

Problem Statement

Design a parameterized D flip-flop with synchronous active-high reset. When `rst = 1` on the rising clock edge, the output resets to 0. Include a synchronous load enable (`en`).

Verilog Solution - dff_sync.v

```

1      // -----
2      // Module : dff_sync
3      // Purpose : Parameterized D-FF, Synchronous Reset + Enable
4      // -----
5      module dff_sync #(
6          parameter WIDTH = 8,
7          parameter RESET_VAL = 0
8      )(
9          input wire      clk,
10         input wire      rst,    // Synchronous, active-high
11         input wire      en,     // Load enable
12         input wire [WIDTH-1:0] d,
13         output reg [WIDTH-1:0] q
14     );
15     always @(posedge clk) begin
16         if (rst)      q <= {WIDTH{1'b0}};    // Sync reset
17         else if (en)  q <= d;                // Load when enabled
18         // else: hold current value (implicit)
19     end
20 endmodule
21

```

Timing Discussion: The critical path runs from `d` through the MUX selecting between `d` and `q` to the D input of the flip-flop. Reset has the same path timing as data. Setup time must hold for all three paths: `rst`, `en`, and `d`.

Common Interview Mistakes

- **Blocking assignment in sequential logic:** Writing `q = d` inside `always @(posedge clk)` instead of `q <= d`. This is the most common RTL interview error worldwide.
- **Async vs sync reset confusion:** Adding `negedge rst` to the sensitivity list converts to asynchronous reset — a completely different circuit.
- **Width mismatch:** Resetting with `q <= 0` (integer) instead of `q <= {WIDTH{1'b0}}` causes simulator warnings.

Pro Tips

- **Sync reset** is preferred in ASIC flows — it avoids clock domain issues and the reset net is not on the critical path.
- **Async reset** is common in FPGA flows because FFs have dedicated async set/reset pins. Know when to use each.
- Always use `<=` (non-blocking) for all flip-flop assignments. This models the actual behavior of physical D flip-flops.

Interviewer's Perspective

Why asked: The flip-flop is the atom of all sequential logic.

Evaluating: Blocking vs non-blocking, sensitivity list correctness, reset style awareness.

Follow-ups: “Async reset version”, “Async reset with sync de-assert”, “Add preset”, “What if WIDTH=1?”

Follow-Up Questions

1. Redesign with asynchronous active-low reset.
2. Design an async reset, synchronous de-assert flip-flop.
3. What is the setup and hold time window?
4. How does the synthesizer implement the enable (`en`) signal?
5. What is clock gating, and how does it relate to flip-flop enable?

Question 5

Design a 4-bit Synchronous Up Counter

BEGINNER

Companies That Ask This Question

Texas Instruments, Intel, Qualcomm, Samsung, MediaTek

Problem Statement

Design a parameterized synchronous binary up counter with synchronous reset, load enable, and a terminal count output (`tc`) that pulses for one clock cycle when the counter reaches its maximum value.

Verilog Solution - counter_up.v

```

1      // -----
2      // Module : counter_up
3      // Purpose : Parameterized Synchronous Up Counter
4      // -----
5      module counter_up #(
6          parameter WIDTH = 4
7      )(
8          input wire      clk,
9          input wire      rst,    // Synchronous reset
10         input wire      en,     // Count enable
11         output reg [WIDTH-1:0] count,
12         output wire      tc     // Terminal count
13     );
14     localparam MAX_VAL = (1 << WIDTH) - 1;
15     // Terminal count: asserted when counter is at maximum
16     assign tc = (count == MAX_VAL[WIDTH-1:0]) & en;
17     always @(posedge clk) begin
18         if (rst)      count <= {WIDTH{1'b0}};
19         else if (en)  count <= count + 1'b1;
20     end
21 endmodule
22

```

Hardware Interpretation: A WIDTH-bit counter synthesizes to WIDTH flip-flops and a ripple-carry adder. The terminal count comparator is a XNOR tree (all bits must equal 1) AND-gated with `en`. **Timing Discussion:** The critical path is the carry propagation through the adder. For a 32-bit counter, this is 32 XOR/AND delays. A carry-lookahead adder reduces this to $O(\log N)$.

Common Interview Mistakes

- Defining `tc` inside `always` block as `reg` and adding a one-cycle delay to terminal count.
- Using `count + 1` (32-bit integer addition) causing width expansion warnings. Always use `count + 1'b1` or `count + {{(WIDTH-1){1'b0}}, 1'b1}`.
- Forgetting that `localparam MAX_VAL` is an integer by default — always slice or cast it to the correct width.

Follow-Up Questions

1. Add a synchronous load input (`load`, `load_val`).
2. Convert to a down counter with borrow output.
3. Design a modulo-N counter (counts 0 to N-1).
4. How do you implement a BCD counter (0 to 9)?
5. What is the maximum clock frequency limited by?

Practice Challenge

Design a modulo-N counter where N is a parameter. The counter should wrap from N-1 back to 0. Include both `tc` (terminal count) and a configurable initial value `INIT`.

Question 6

Design a Universal Shift Register

BEGINNER

Companies That Ask This Question

Intel, NVIDIA, AMD, Broadcom, ARM

Problem Statement

Design a parameterized universal shift register that supports four modes via a 2-bit control: 00=Hold, 01=Shift-Right, 10=Shift-Left, 11=Parallel Load.

Verilog Solution - shift_reg_universal.v

```

1      //-----
2      // Module : shift_reg_universal
3      // Purpose : Universal Shift Register (4 modes)
4      //-----
5      module shift_reg_universal #(
6          parameter WIDTH = 8
7      )(
8          input wire      clk,
9          input wire      rst,
10         input wire [1:0] ctrl,      // Mode select
11         input wire [WIDTH-1:0] d_parallel, // Parallel data in
12         input wire      d_serial_r, // Serial in (right)
13         input wire      d_serial_l, // Serial in (left)
14         output reg [WIDTH-1:0] q,
15         output wire      q_serial_r, // LSB serial out
16         output wire      q_serial_l // MSB serial out
17     );
18     localparam HOLD      = 2'b00;
19     localparam SHIFT_RIGHT = 2'b01;
20     localparam SHIFT_LEFT  = 2'b10;
21     localparam PARALLEL_LD = 2'b11;
22     assign q_serial_r = q[0];      // Serial out for right shift
23     assign q_serial_l = q[WIDTH-1]; // Serial out for left shift
24     always @(posedge clk) begin
25         if (rst) begin
26             q <= {WIDTH{1'b0}};
27         end else begin
28             case (ctrl)
29                 HOLD:      q <= q;          // Hold
30                 SHIFT_RIGHT: q <= {d_serial_r, q[WIDTH-1:1]}; // Shift right
31                 SHIFT_LEFT:  q <= {q[WIDTH-2:0], d_serial_l}; // Shift left
32                 PARALLEL_LD: q <= d_parallel; // Parallel load
33             endcase
34         end
35     end
36 endmodule
37

```


Common Interview Mistakes

- Shift direction confusion: Right shift inserts from MSB (`{d_serial_r, q[N-1:1]}`), not LSB. Sketch the bit positions before coding.
- Forgetting to expose serial outputs — interviewers always ask for these.
- Omitting the `HOLD` case without a `default` — creates a latch.

Follow-Up Questions

1. Add a rotate-right and rotate-left mode (ctrl becomes 3-bit).
2. Use this as a PISO (parallel-in, serial-out) converter.
3. Implement a LFSR (Linear Feedback Shift Register) using this module.
4. How does a barrel shifter differ from this register?
5. Design a ring counter using the shift-right mode.

Question 7

Design a Parameterized Comparator

BEGINNER

Companies That Ask This Question

Intel, Texas Instruments, Samsung, Marvell, Synopsys

Problem Statement

Design a parameterized magnitude comparator. Given inputs **a** and **b**, output three flags: **eq** ($a==b$), **gt** ($a>b$), **lt** ($a<b$). Support both unsigned and signed comparison via a parameter.

Verilog Solution - comparator.v

```

1      // -----
2      // Module : comparator
3      // Purpose : Parameterized Magnitude Comparator
4      // -----
5      module comparator #(
6          parameter WIDTH = 8,
7          parameter SIGNED = 0    // 1 = signed, 0 = unsigned
8      )(
9          input wire [WIDTH-1:0] a,
10         input wire [WIDTH-1:0] b,
11         output wire          eq,    // a == b
12         output wire          gt,    // a > b
13         output wire          lt     // a < b
14     );
15     generate
16     if (SIGNED) begin : signed_cmp
17         wire signed [WIDTH-1:0] sa = a;
18         wire signed [WIDTH-1:0] sb = b;
19         assign eq = (sa == sb);
20         assign gt = (sa > sb);
21         assign lt = (sa < sb);
22     end else begin : unsigned_cmp
23         assign eq = (a == b);
24         assign gt = (a > b);
25         assign lt = (a < b);
26     end
27     endgenerate
28     endmodule
29 
```

Pro Tips

The **generate** block is the right way to handle *compile-time* parameter-driven structural differences. The synthesizer will completely eliminate the unused branch. Signed comparison on unsigned **reg/wire** is achieved with the **\$signed()** cast.

Follow-Up Questions

1. How does signed comparison differ in two's complement arithmetic?
2. Design a comparator using only subtraction.
3. What is the gate-level implementation for `eq`?
4. Build a 32-bit comparator from two 16-bit comparators.
5. When does `gt = !lt & !eq` hold, and when does it not?

Question 8

Design a Moore FSM — Sequence Detector (101)

BEGINNER

Companies That Ask This Question

Intel, Qualcomm, NVIDIA, AMD, Texas Instruments, ARM

Problem Statement

Design a Moore-style FSM that detects the serial bit sequence **101**. The output **detected** is asserted for one cycle after the complete sequence is observed. Support overlapping detection.

Verilog Solution - seq_detect_101.v

```

1      // -----
2      // Module : seq_detect_101 (Moore FSM)
3      // Purpose : Detects overlapping sequence "101"
4      // -----
5      module seq_detect_101 (
6          input wire clk,
7          input wire rst,
8          input wire din,          // Serial bit input
9          output reg detected      // Pulse when "101" detected
10     );
11     // State encoding
12     typedef enum reg [1:0] {
13         S0 = 2'b00, // Initial / reset state
14         S1 = 2'b01, // Got "1"
15         S2 = 2'b10, // Got "10"
16     } state_t;
17     state_t curr_state, next_state;
18     // State register
19     always @(posedge clk) begin
20         if (rst) curr_state <= S0;
21         else curr_state <= next_state;
22     end
23     // Next state logic
24     always @(*) begin
25         case (curr_state)
26             S0: next_state = din ? S1 : S0;
27             S1: next_state = din ? S1 : S2;
28             S2: next_state = din ? S3 : S0;
29             S3: next_state = din ? S1 : S2; // Overlap: "1" from "101" counts
30             default: next_state = S0;
31         endcase
32     end
33     // Output logic (Moore: depends only on current state)
34     always @(*) begin
35         detected = (curr_state == S3);
36     end
37 endmodule
38
39

```

RTL Design Explanation: Three separate always blocks implement the canonical 3-block FSM style: (1) state register, (2) next-state logic, (3) output logic. This is the industry-standard

approach for FSMs.

Common Interview Mistakes

- **2-block vs 3-block FSM:** Merging next-state and output in one block works but is harder to review and synthesize cleanly.
- **Moore vs Mealy confusion:** Moore output depends only on state. Mealy output also depends on the input — mention this distinction.
- **No overlap:** Going back to S0 after detection misses the overlapping case. The interviewer will test the edge case “101101”.

Interviewer’s Perspective

Why asked: FSMs are the backbone of controllers, protocol decoders, and parsers.

Evaluating: State diagram correctness, coding style, Moore/Mealy distinction, reset handling, overlap handling.

Impress: Draw the state diagram first, then write code. Interviewers love this.

Follow-Up Questions

1. Convert this to a Mealy FSM. How does the output timing differ?
2. Detect the sequence “1011” with overlapping.
3. What happens if `rst` is asynchronous?
4. How would you verify this FSM with a testbench?
5. Implement this FSM with one-hot encoding.

Question 9

Design a Parity Generator and Checker

BEGINNER

Companies That Ask This Question

Intel, Texas Instruments, Synopsys, Marvell, Samsung

Problem Statement

Design a parameterized even/odd parity generator. Also design a parity checker that asserts an **error** flag when received data parity does not match the expected parity bit.

Verilog Solution - parity.v

```

1      // -----
2      // Module : parity_gen
3      // Purpose : Parameterized Even/Odd Parity Generator
4      // -----
5      module parity_gen #(
6          parameter WIDTH      = 8,
7          parameter ODD_PARITY = 0 // 0=even, 1=odd
8      )(
9          input  wire [WIDTH-1:0] data,
10         output wire          parity_out
11     );
12     wire even_parity;
13     assign even_parity = ^data; // XOR reduction = even parity bit
14     assign parity_out  = ODD_PARITY ? ~even_parity : even_parity;
15     endmodule
16     // -----
17     // Module : parity_chk
18     // Purpose : Parity Checker
19     // -----
20     module parity_chk #(
21         parameter WIDTH      = 8,
22         parameter ODD_PARITY = 0
23     )(
24         input  wire [WIDTH-1:0] data,
25         input  wire          parity_in, // Received parity bit
26         output wire          error     // High if parity mismatch
27     );
28     wire expected;
29     wire even_p = ^data;
30     assign expected = ODD_PARITY ? ~even_p : even_p;
31     assign error    = (parity_in != expected);
32     endmodule
33

```

Pro Tips

The XOR reduction operator `^data` is one of Verilog's most elegant constructs. It computes the XOR of all bits in one statement. In hardware, this is a tree of XOR gates with $O(\log N)$ depth.

Follow-Up Questions

1. What parity errors can parity *not* detect?
2. How many error-detecting bits are needed for a Hamming code?
3. Implement a 2D parity check (row and column parity).
4. How does parity differ from CRC?
5. Design a SECDED (Single Error Correct, Double Error Detect) ECC.

Question 10

Design a Clock Divider (Divide-by-N)

BEGINNER

Companies That Ask This Question

NVIDIA, AMD, Intel, Qualcomm, Apple, Broadcom

Problem Statement

Design a parameterized clock divider that produces an output clock `clk_out` at $1/N$ of the input clock frequency. For even N , the output should have 50% duty cycle. The module should be synthesizable.

Verilog Solution - clk_divider.v

```

1      // -----
2      // Module   : clk_divider
3      // Purpose  : Parameterized Clock Divider (even N, 50% duty)
4      // Note     : For use as functional reference; in production,
5      //             use dedicated clock buffers / PLL primitives.
6      // -----
7      module clk_divider #(
8          parameter DIV = 4    // Must be even and >= 2
9      )(
10         input  wire clk,
11         input  wire rst,
12         output reg  clk_out
13     );
14     localparam HALF = DIV / 2;
15     reg [$clog2(DIV):0] count; // Counter wide enough for DIV
16     always @(posedge clk) begin
17         if (rst) begin
18             count <= 0;
19             clk_out <= 1'b0;
20         end else begin
21             if (count == HALF - 1) begin
22                 clk_out <= ~clk_out; // Toggle every HALF cycles
23                 count <= 0;
24             end else begin
25                 count <= count + 1'b1;
26             end
27         end
28     end
29 endmodule
30

```

Hardware Interpretation: This synthesizes to a counter plus a T-flip-flop (the toggle of `clk_out`). The counter is the modulo-HALF counter with a terminal compare.

Important Note for Interviews

Interviewers will ask: "Can this clock drive flip-flops?" The correct answer is: **Only with a clock buffer**. In ASIC flows, derived clocks must go through a clock tree. In FPGA flows, use BUFG primitives. Never drive flip-flop clocks directly from `reg` outputs in production

designs.

Common Interview Mistakes

- Generating a non-50% duty cycle clock (counting to N, toggling once).
- Not using `$clog2` — hard-coded counter width limits parameterization.
- Forgetting that derived clocks need clock buffers in real designs.

Follow-Up Questions

1. How do you divide by an odd number and maintain 50% duty cycle?
2. What is the difference between a clock divider and a clock gating cell?
3. How does a PLL generate divided/multiplied clocks?
4. What CDC issues arise when crossing from `clk` to `clk_out` domains?
5. Implement divide-by-3 with 50% duty cycle (requires both edges).

Intermediate Questions

Chapter Overview

These 20 questions cover the core design blocks expected in every RTL interview. Topics: FIFOs, ALUs, FSMs, RAMs, UART, SPI, Arbiters, PWM, Synchronizers, Pipeline Registers, Gray Counters, CRC, and protocol interfaces.

Question 11

Design a Parameterized Synchronous FIFO

INTERMEDIATE

Companies That Ask This Question

NVIDIA, AMD, Intel, Qualcomm, Apple, Broadcom, ARM

Problem Statement

Design a parameterized synchronous FIFO (single clock domain). Include: `wr_en`, `rd_en`, `full`, `empty`, `count` outputs. Both read and write are registered (synchronous). Depth and data width are parameters. Handle simultaneous read/write correctly.

Interview Requirements: Correct pointer arithmetic, full/empty flag generation without a race condition, and power-of-2 depth for natural wrap.

Verilog Solution - sync_fifo.v

```

1      // -----
2      // Module : sync_fifo
3      // Purpose : Parameterized Synchronous FIFO
4      // -----
5      module sync_fifo #(
6          parameter DATA_WIDTH = 8,
7          parameter DEPTH        = 16 // Must be power of 2
8      )(
9          input  wire          clk,
10         input  wire          rst,
11         // Write port
12         input  wire          wr_en,
13         input  wire [DATA_WIDTH-1:0] wr_data,
14         output wire          full,
15         // Read port
16         input  wire          rd_en,
17         output reg [DATA_WIDTH-1:0] rd_data,
18         output wire          empty,
19         // Status
20         output wire [$clog2(DEPTH):0] count
21     );
22     localparam PTR_WIDTH = $clog2(DEPTH);
23     // Memory array
24     reg [DATA_WIDTH-1:0] mem [0:DEPTH-1];
25
26     reg [PTR_WIDTH:0] wr_ptr;
27     reg [PTR_WIDTH:0] rd_ptr;
28     // Status flags
29     assign empty = (wr_ptr == rd_ptr);
30     assign full  = (wr_ptr[PTR_WIDTH] != rd_ptr[PTR_WIDTH]) &&
31     (wr_ptr[PTR_WIDTH-1:0] == rd_ptr[PTR_WIDTH-1:0]);
32     assign count = wr_ptr - rd_ptr;
33     // Write logic
34     always @(posedge clk) begin
35         if (rst) begin
36             wr_ptr <= 0;
37         end else if (wr_en && !full) begin

```

```

38     mem[wr_ptr[PTR_WIDTH-1:0]] <= wr_data;
39     wr_ptr <= wr_ptr + 1'b1;
40     end
41     end
42     // Read logic
43     always @(posedge clk) begin
44         if (rst) begin
45             rd_ptr <= 0;
46             rd_data <= {DATA_WIDTH{1'b0}};
47         end else if (rd_en && !empty) begin
48             rd_data <= mem[rd_ptr[PTR_WIDTH-1:0]];
49             rd_ptr <= rd_ptr + 1'b1;
50         end
51     end
52     endmodule
53

```

Line-by-Line Explanation:

- **Extra pointer bit:** `wr_ptr` and `rd_ptr` are `PTR_WIDTH+1` bits wide. The MSB differentiates full from empty when lower bits are equal.
- **Full condition:** MSBs differ AND lower bits equal $\rightarrow$ the write pointer has wrapped past the read pointer by exactly one full revolution.
- **Empty condition:** Pointers are identical.
- **Count:** Simple subtraction — works because unsigned wraparound gives the correct occupancy for power-of-2 depths.

Timing Discussion: Critical paths: (a) write data path from `wr_data` to `mem` array; (b) read data path from `mem` to `rd_data`; (c) count comparator for `full/empty` flags.

Common Interview Mistakes

- **Full/Empty flag collision:** Using only `PTR_WIDTH`-bit pointers. When empty, lower bits are equal, but when full, lower bits are also equal. Without the extra MSB, you cannot distinguish the two states.
- **Non-power-of-2 depth:** Using `depth=10` breaks the wraparound arithmetic. Always use power-of-2, or add explicit modulo logic.
- **Simultaneous R/W:** Forgetting to handle `wr_en` & `rd_en` simultaneously. If full and both are active, write should be blocked but read can proceed (count decreases).
- **Forgetting FWFT:** Some interviewers ask for First-Word-Fall-Through (FWFT) behavior where `rd_data` is valid immediately after write without a read enable.

Pro Tips

- For **asynchronous FIFO** (different clock domains), replace the pointers with Gray-coded pointers and use two-flop synchronizers at each boundary. This is one of the most asked advanced questions (see Q32).
- Add `almost_full` and `almost_empty` flags with programmable thresholds — this is a standard DMA back-pressure feature.
- `$clog2(DEPTH)` is a synthesis-safe system function for computing log base 2. It is available in Verilog-2001 and later.

Interviewer's Perspective

Why asked: FIFOs appear in virtually every SoC — memory interfaces, DMA, network buffers, protocol bridges.

Evaluating: Full/empty flag correctness, pointer arithmetic, simultaneous R/W, synthesizability.

Follow-ups: “Make it async FIFO”, “Add almost-full flag”, “What is FWFT?”, “How do you flush a FIFO?”

Impress: Immediately explain the extra MSB pointer trick before writing a single line of code.

Follow-Up Questions

1. Convert this to an asynchronous FIFO using Gray-coded pointers.
2. Add programmable almost-full and almost-empty thresholds.
3. Implement FWFT (First-Word-Fall-Through) behavior.
4. How do you flush/reset one side of an async FIFO independently?
5. What is the minimum FIFO depth needed to absorb a burst of 8 writes at 2x the read rate?

Question 12

Design a Barrel Shifter

INTERMEDIATE

Companies That Ask This Question

Intel, AMD, ARM, Qualcomm, Apple

Problem Statement

Design a parameterized barrel shifter supporting logical shift left, logical shift right, arithmetic shift right, and rotate left/right. The shift amount is variable. All operations must be combinational with zero propagation delay (no clock).

Verilog Solution - barrel_shifter.v

```

1  // -----
2  // Module : barrel_shifter
3  // Purpose : Parameterized Barrel Shifter
4  // op: 000=LSL, 001=LSR, 010=ASR, 011=ROL, 100=ROR
5  // -----
6  module barrel_shifter #(
7      parameter WIDTH = 32
8  )(
9      input wire [WIDTH-1:0]    data_in,
10     input wire [$clog2(WIDTH)-1:0] shift_amt,
11     input wire [2:0]          op,
12     output reg [WIDTH-1:0]    data_out
13 );
14     localparam LSL = 3'b000; // Logical Shift Left
15     localparam LSR = 3'b001; // Logical Shift Right
16     localparam ASR = 3'b010; // Arithmetic Shift Right
17     localparam ROL = 3'b011; // Rotate Left
18     localparam ROR = 3'b100; // Rotate Right
19     always @(*) begin
20         case (op)
21             LSL: data_out = data_in << shift_amt;
22             LSR: data_out = data_in >> shift_amt;
23             ASR: data_out = $signed(data_in) >>> shift_amt; // Sign-extend
24             ROL: data_out = (data_in << shift_amt) |
25                 (data_in >> (WIDTH - shift_amt));
26             ROR: data_out = (data_in >> shift_amt) |
27                 (data_in << (WIDTH - shift_amt));
28             default: data_out = data_in;
29         endcase
30     end
31 endmodule
32

```

Hardware Interpretation: A barrel shifter is implemented as a cascade of $\log_2(WIDTH)$ mux stages. Each stage selects between the original and a fixed power-of-2 shifted version. For a 32-bit shifter, this is 5 stages of 32 2:1 muxes. Total delay: $5 \times T_{mux}$.

Common Interview Mistakes

- **ASR with unsigned type:** Using `data_in >> shift_amt` without the `$signed()` cast results in a logical right shift (zero-fill), not arithmetic (sign-fill).
- **Rotation overflow:** For ROL, if `shift_amt = 0`, the expression `data_in << (WIDTH - 0)` equals `data_in << WIDTH`, which is undefined behavior in Verilog. Add a guard:
`shift_amt == 0 ? data_in : ....`

Follow-Up Questions

1. Implement the barrel shifter as a cascaded mux structure explicitly.
2. What is the critical path timing for a 64-bit barrel shifter?
3. How does an ARM processor implement the barrel shifter in its ALU?
4. Design a funnel shifter (used in unaligned memory access).
5. What is the difference between `>` and `>>` in Verilog?

Question 13

Design a Simple ALU

INTERMEDIATE

Companies That Ask This Question

Intel, AMD, ARM, Qualcomm, Apple, NVIDIA

Problem Statement

Design a parameterized ALU supporting: ADD, SUB, AND, OR, XOR, NOT, SHL, SHR. Include carry-out, overflow, zero, and negative flags (like a real CPU ALU).

Verilog Solution - alu.v

```

1      // -----
2      // Module : alu
3      // Purpose : Parameterized ALU with Status Flags
4      // -----
5      module alu #(
6          parameter WIDTH = 16
7      )(
8          input wire [WIDTH-1:0] a,
9          input wire [WIDTH-1:0] b,
10         input wire [3:0] op,          // Operation select
11         output reg [WIDTH-1:0] result,
12         output reg          carry,    // Carry out
13         output reg          overflow, // Signed overflow
14         output wire          zero,    // Result is zero
15         output wire          negative // Result MSB (sign bit)
16     );
17     // Op codes
18     localparam ADD = 4'h0;
19     localparam SUB = 4'h1;
20     localparam AND = 4'h2;
21     localparam OR  = 4'h3;
22     localparam XOR = 4'h4;
23     localparam NOT = 4'h5;
24     localparam SHL = 4'h6;
25     localparam SHR = 4'h7;
26     localparam SLT = 4'h8; // Set less than (signed)
27     wire [WIDTH:0] add_result = {1'b0, a} + {1'b0, b};
28     wire [WIDTH:0] sub_result = {1'b0, a} - {1'b0, b};
29     assign zero      = (result == {WIDTH{1'b0}});
30     assign negative  = result[WIDTH-1];
31     always @(*) begin
32         carry      = 1'b0;
33         overflow    = 1'b0;
34         case (op)
35             ADD: begin
36                 result  = add_result[WIDTH-1:0];
37                 carry   = add_result[WIDTH];
38                 // Signed overflow: (+)+(+)=(-) or (-)+(-)=(+)
39                 overflow = (~a[WIDTH-1] & ~b[WIDTH-1] & result[WIDTH-1]) |
40                     ( a[WIDTH-1] &  b[WIDTH-1] & ~result[WIDTH-1]);
41             end
42             SUB: begin

```

```

43     result    = sub_result[WIDTH-1:0];
44     carry     = sub_result[WIDTH]; // Borrow
45     overflow = (~a[WIDTH-1] & b[WIDTH-1] & result[WIDTH-1]) |
46               ( a[WIDTH-1] & ~b[WIDTH-1] & ~result[WIDTH-1]);
47     end
48     AND: result = a & b;
49     OR:  result = a | b;
50     XOR: result = a ^ b;
51     NOT: result = ~a;
52     SHL: result = a << b[$clog2(WIDTH)-1:0];
53     SHR: result = a >> b[$clog2(WIDTH)-1:0];
54     SLT: result = ($signed(a) < $signed(b)) ? {{WIDTH-1{1'b0}},1'b1}
55           : {WIDTH{1'b0}};
56     default: result = {WIDTH{1'b0}};
57     endcase
58     end
59     endmodule
60

```

Common Interview Mistakes

- **Overflow detection:** This is the most missed part. Signed overflow occurs when two positives yield a negative or two negatives yield a positive. The formula is: `overflow = (a[N-1] == b[N-1]) & (result[N-1] != a[N-1])`.
- **Carry vs overflow:** Carry is relevant for unsigned arithmetic; overflow is relevant for signed arithmetic. Know the difference.
- **Shift amount width:** Using the full `b` as shift amount causes redundant bits. Slice it to `$clog2(WIDTH)` bits.

Follow-Up Questions

1. Add a multiply operation (partial product accumulation).
2. How does the ARM Thumb ALU differ from a standard ALU?
3. Implement a MAC (Multiply-Accumulate) unit using this ALU.
4. What is the critical path of a 32-bit adder?
5. How do CPUs implement SIMD operations at the ALU level?

Question 14

Design a Mealy FSM — Traffic Light Controller

INTERMEDIATE

Companies That Ask This Question

Texas Instruments, Intel, Qualcomm, Synopsys, Cadence

Problem Statement

Design a traffic light controller for a two-road intersection (North-South and East-West). The controller should cycle through: Green → Yellow → Red. Include a timer input `timer_done` that triggers state transitions. Both directions must never be green simultaneously (safety constraint).

Verilog Solution - traffic_light.v

```

1  // -----
2  // Module : traffic_light
3  // Purpose : Two-road Traffic Light Controller (Moore FSM)
4  // Outputs: ns_light[1:0], ew_light[1:0]
5  //          2'b00=RED, 2'b01=YELLOW, 2'b10=GREEN
6  // -----
7  module traffic_light (
8      input wire      clk,
9      input wire      rst,
10     input wire      timer_done, // Timer expired
11     output reg [1:0] ns_light,  // N-S direction
12     output reg [1:0] ew_light  // E-W direction
13 );
14 localparam RED      = 2'b00;
15 localparam YELLOW   = 2'b01;
16 localparam GREEN    = 2'b10;
17 // States
18 typedef enum reg [2:0] {
19     NS_GREEN = 3'd0, // N-S=Green, E-W=Red
20     NS_YELLOW = 3'd1, // N-S=Yellow, E-W=Red
21     EW_GREEN = 3'd2, // N-S=Red, E-W=Green
22     EW_YELLOW = 3'd3 // N-S=Red, E-W=Yellow
23 } state_t;
24 state_t state, next_state;
25 // State register
26 always @(posedge clk) begin
27     if (rst) state <= NS_GREEN;
28     else state <= next_state;
29 end
30 // Next-state logic
31 always @(*) begin
32     next_state = state;
33     if (timer_done) begin
34         case (state)
35             NS_GREEN: next_state = NS_YELLOW;
36             NS_YELLOW: next_state = EW_GREEN;
37             EW_GREEN: next_state = EW_YELLOW;
38             EW_YELLOW: next_state = NS_GREEN;
39             default: next_state = NS_GREEN;

```

```
40     endcase
41     end
42     end
43     // Output logic (Moore)
44     always @(*) begin
45         case (state)
46             NS_GREEN: begin ns_light = GREEN; ew_light = RED; end
47             NS_YELLOW: begin ns_light = YELLOW; ew_light = RED; end
48             EW_GREEN: begin ns_light = RED; ew_light = GREEN; end
49             EW_YELLOW: begin ns_light = RED; ew_light = YELLOW; end
50             default: begin ns_light = RED; ew_light = RED; end
51         endcase
52     end
53 endmodule
54
```

Common Interview Mistakes

- **Safety violation:** Both directions going green simultaneously. Always ensure the `default` output is all-RED.
- **Missing timer integration:** Transitioning every clock cycle without a timer input is not realistic.
- **Glitch on outputs:** Combinational output from an FSM can glitch during state transitions. Register the outputs if glitch-free is required.

Follow-Up Questions

1. Add a pedestrian request input that extends the current green phase.
2. Add an emergency vehicle input that immediately forces all-red.
3. How would you implement the countdown timer as a separate module?
4. Design a 4-way intersection controller.
5. How do you formally verify that both lights are never green simultaneously?

Question 15

Design a Single-Port Synchronous RAM

INTERMEDIATE

Companies That Ask This Question

Intel, AMD, NVIDIA, Qualcomm, Synopsys, Cadence

Problem Statement

Design a parameterized single-port synchronous SRAM (read-first or write-first configurable). Include byte-enable write support. The module should be synthesizable to block RAM on FPGAs and standard cell RAM in ASIC flows.

Verilog Solution - spram.v

```

1      // -----
2      // Module : spram
3      // Purpose : Single-Port Synchronous RAM with Byte Enable
4      // -----
5      module spram #(
6          parameter DEPTH      = 256,
7          parameter DATA_WIDTH = 32,
8          parameter BYTES      = DATA_WIDTH / 8 // Byte enables
9      )(
10         input  wire          clk,
11         input  wire          we,           // Write enable
12         input  wire [BYTES-1:0] be,       // Byte enables
13         input  wire [$clog2(DEPTH)-1:0] addr,
14         input  wire [DATA_WIDTH-1:0] din,
15         output reg [DATA_WIDTH-1:0] dout
16     );
17     reg [DATA_WIDTH-1:0] mem [0:DEPTH-1];
18     // Write: byte-granular
19     always @(posedge clk) begin
20         if (we) begin : write_block
21             integer i;
22             for (i = 0; i < BYTES; i = i + 1) begin
23                 if (be[i]) begin
24                     mem[addr][i*8 +: 8] <= din[i*8 +: 8];
25                 end
26             end
27         end
28     end
29     // Read (Read-First: old data on same-cycle read/write)
30     always @(posedge clk) begin
31         dout <= mem[addr];
32     end
33 endmodule
34

```

Pro Tips

- **Read-First vs Write-First:** In read-first mode (`dout <= mem[addr]` before write), a simultaneous read and write to the same address returns the old value. In write-first, the

new value is returned. Most FPGA block RAMs support both via configuration bits.

- **Part-select with `+:`** The expression `din[i*8 +: 8]` selects 8 bits starting at bit `i*8`. This is an indexed part-select (Verilog-2001) and is the cleanest way to handle byte lanes.
- For ASIC flows, the actual SRAM is generated by a memory compiler (e.g., ARM Artisan, Virage) and only the wrapper RTL is written.

Follow-Up Questions

1. Convert to a dual-port RAM (simultaneous read/write ports).
2. How does a true dual-port RAM handle simultaneous write to the same address?
3. What is a ROM, and how is it initialized in Verilog (`$readmemb`)?
4. How does a register file differ from a general SRAM?
5. Describe how an FPGA BRAM is configured for different aspect ratios.

Question 16

Design a Register File

INTERMEDIATE

Companies That Ask This Question

ARM, NVIDIA, AMD, Intel, Qualcomm, Apple

Problem Statement

Design a 32-register, parameterized register file with two read ports and one write port. Register 0 must always read as zero (hardwired to 0, as in RISC-V). Write is synchronous; reads are asynchronous (combinational).

Verilog Solution - regfile.v

```

1      // -----
2      // Module : regfile
3      // Purpose : RISC-V-style 32-entry Register File
4      // -----
5      module regfile #(
6          parameter DATA_WIDTH = 32,
7          parameter ADDR_WIDTH = 5    // 2^5 = 32 registers
8      )(
9          input wire          clk,
10         // Write port
11         input wire          wr_en,
12         input wire [ADDR_WIDTH-1:0] wr_addr,
13         input wire [DATA_WIDTH-1:0] wr_data,
14         // Read port 1
15         input wire [ADDR_WIDTH-1:0] rd_addr1,
16         output wire [DATA_WIDTH-1:0] rd_data1,
17         // Read port 2
18         input wire [ADDR_WIDTH-1:0] rd_addr2,
19         output wire [DATA_WIDTH-1:0] rd_data2
20     );
21     reg [DATA_WIDTH-1:0] regs [0:(1<<ADDR_WIDTH)-1];
22     // Synchronous write (x0 protected: never written)
23     always @(posedge clk) begin
24         if (wr_en && wr_addr != {ADDR_WIDTH{1'b0}}) begin
25             regs[wr_addr] <= wr_data;
26         end
27     end
28     // Asynchronous read (combinational)
29     // x0 is hardwired to zero
30     assign rd_data1 = (rd_addr1 == {ADDR_WIDTH{1'b0}}) ?
31     {DATA_WIDTH{1'b0}} : regs[rd_addr1];
32     assign rd_data2 = (rd_addr2 == {ADDR_WIDTH{1'b0}}) ?
33     {DATA_WIDTH{1'b0}} : regs[rd_addr2];
34     endmodule
35

```

Interviewer's Perspective

Why asked: The register file is the heart of every CPU pipeline. Understanding it demonstrates CPU microarchitecture knowledge.

Evaluating: x0 handling, sync write / async read, multi-port correctness, write-before-read (forwarding).

Follow-ups: “Add forwarding for same-cycle write-read”, “Add a write mask”, “How does a 3R2W register file synthesize?”

Follow-Up Questions

1. Add write-to-read forwarding: if `rd_addr1 == wr_addr` and `wr_en`, return `wr_data` directly.
2. Design a 3-read-port, 2-write-port register file.
3. How does a multi-ported register file synthesize in ASIC?
4. What is register renaming, and why does it require more than 32 physical registers?
5. How do you initialize a register file for simulation?

Question 17

Design a UART Transmitter

INTERMEDIATE

Companies That Ask This Question

Texas Instruments, Intel, Qualcomm, Samsung, Marvell, MediaTek

Problem Statement

Design a UART transmitter supporting configurable baud rate via a clock divider parameter. The UART frame: 1 start bit, 8 data bits (LSB first), 1 stop bit. Include a `tx_busy` signal and a single-cycle `tx_start` pulse.

Verilog Solution - uart_tx.v

```

1  // -----
2  // Module : uart_tx
3  // Purpose : UART Transmitter (8N1 format)
4  // CLK_FREQ/BAUD must be an integer (e.g., 50MHz/115200)
5  // -----
6  module uart_tx #(
7      parameter CLK_FREQ = 50_000_000,
8      parameter BAUD      = 115_200
9  )(
10     input wire      clk,
11     input wire      rst,
12     input wire      tx_start,    // Pulse to start transmission
13     input wire [7:0] tx_data,    // Byte to transmit
14     output reg       tx,          // Serial output (idle=1)
15     output reg       tx_busy     // High during transmission
16 );
17     localparam BAUD_DIV = CLK_FREQ / BAUD - 1;
18     localparam IDLE     = 2'd0;
19     localparam START    = 2'd1;
20     localparam DATA    = 2'd2;
21     localparam STOP     = 2'd3;
22     reg [1:0] state;
23     reg [15:0] baud_cnt;
24     reg [2:0] bit_idx;
25     reg [7:0] shift_reg;
26     wire baud_tick = (baud_cnt == 16'h0);
27     // Baud rate generator
28     always @(posedge clk) begin
29         if (rst || baud_cnt == 0)
30             baud_cnt <= BAUD_DIV[15:0];
31         else
32             baud_cnt <= baud_cnt - 1'b1;
33     end
34     // Transmit FSM
35     always @(posedge clk) begin
36         if (rst) begin
37             state <= IDLE;
38             tx <= 1'b1;    // Idle high
39             tx_busy <= 1'b0;
40             bit_idx <= 3'd0;

```

```

41     end else begin
42     case (state)
43     IDLE: begin
44         tx      <= 1'b1;
45         tx_busy <= 1'b0;
46         if (tx_start) begin
47             shift_reg <= tx_data;
48             state     <= START;
49             tx_busy   <= 1'b1;
50         end
51     end
52     START: begin
53         if (baud_tick) begin
54             tx      <= 1'b0;      // Start bit
55             state <= DATA;
56             bit_idx <= 3'd0;
57         end
58     end
59     DATA: begin
60         if (baud_tick) begin
61             tx      <= shift_reg[0];      // LSB first
62             shift_reg <= {1'b0, shift_reg[7:1]}; // Shift right
63             if (bit_idx == 3'd7) state <= STOP;
64             else bit_idx <= bit_idx + 1'b1;
65         end
66     end
67     STOP: begin
68         if (baud_tick) begin
69             tx      <= 1'b1;      // Stop bit
70             state <= IDLE;
71         end
72     end
73     endcase
74     end
75     end
76 endmodule
77

```

Common Interview Mistakes

- **Baud tick timing:** Sampling data at the wrong baud tick boundary causes framing errors. The transmitter should change `tx` on the baud tick.
- **MSB vs LSB first:** UART is always LSB-first. Confirm this in the interview requirements.
- **Idle state:** `tx` must be held high during idle. A logic-0 idle line looks like a continuous start bit.
- **tx_start pulse width:** If `tx_start` is held high, the TX may not restart correctly. Only respond to the first cycle of assertion.

Follow-Up Questions

1. Design the UART receiver to complement this transmitter.
2. Add a TX FIFO to buffer multiple bytes.
3. How do you handle baud rate mismatch between TX and RX?
4. Add parity support (even/odd parity bit between data and stop).
5. How does UART differ from SPI and I2C?

Question 18

Design an SPI Master

INTERMEDIATE

Companies That Ask This Question

Texas Instruments, Intel, Qualcomm, Samsung, Marvell, Broadcom

Problem Statement

Design an SPI Master controller supporting CPOL=0, CPHA=0 (Mode 0). The master generates `sclk`, drives `mosi`, and samples `miso`. Support configurable transfer width (8 or 16 bits). Include `cs_n` (chip select, active low) and a `done` flag.

Verilog Solution - spi_master.v

```

1  // -----
2  // Module : spi_master
3  // Purpose : SPI Master (Mode 0: CPOL=0, CPHA=0)
4  // -----
5  module spi_master #(
6      parameter CLK_DIV    = 4,    // SPI clock = sys_clk / (2*CLK_DIV)
7      parameter DATA_WIDTH = 8
8  )(
9      input  wire          clk,
10     input  wire          rst,
11     input  wire          start,    // Begin transfer
12     input  wire [DATA_WIDTH-1:0] tx_data,
13     output reg [DATA_WIDTH-1:0] rx_data,
14     output reg           done,    // Transfer complete
15     output reg           busy,
16     // SPI pins
17     output reg           sclk,
18     output reg           mosi,
19     input  wire          miso,
20     output reg           cs_n
21 );
22 localparam IDLE      = 2'd0;
23 localparam TRANSFER  = 2'd1;
24 localparam FINISH    = 2'd2;
25 reg [1:0] state;
26 reg [$clog2(CLK_DIV):0] div_cnt;
27 reg [$clog2(DATA_WIDTH):0] bit_cnt;
28 reg [DATA_WIDTH-1:0] shift_tx, shift_rx;
29 reg sclk_rise, sclk_fall;
30 always @(posedge clk) begin
31     if (rst) begin
32         state <= IDLE;
33         sclk <= 1'b0;
34         cs_n <= 1'b1;
35         done <= 1'b0;
36         busy <= 1'b0;
37         div_cnt <= 0;
38     end else begin
39         done <= 1'b0; // Default
40         sclk_rise <= 1'b0;

```

```

41     sclk_fall <= 1'b0;
42     // Clock divider
43     if (state == TRANSFER) begin
44         if (div_cnt == CLK_DIV - 1) begin
45             div_cnt <= 0;
46             sclk <= ~sclk;
47             sclk_rise <= ~sclk; // Rising: sclk was 0, now 1
48             sclk_fall <= sclk; // Falling: sclk was 1, now 0
49         end else begin
50             div_cnt <= div_cnt + 1'b1;
51         end
52     end
53     case (state)
54     IDLE: begin
55         sclk <= 1'b0; cs_n <= 1'b1; busy <= 1'b0;
56         if (start) begin
57             shift_tx <= tx_data;
58             bit_cnt <= DATA_WIDTH;
59             cs_n <= 1'b0;
60             busy <= 1'b1;
61             state <= TRANSFER;
62             div_cnt <= 0;
63         end
64     end
65     TRANSFER: begin
66         if (sclk_fall) begin
67             // Drive MOSI on falling edge (CPHA=0)
68             mosi <= shift_tx[DATA_WIDTH-1]; // MSB first
69             shift_tx <= {shift_tx[DATA_WIDTH-2:0], 1'b0};
70         end
71         if (sclk_rise) begin
72             // Sample MISO on rising edge
73             shift_rx <= {shift_rx[DATA_WIDTH-2:0], miso};
74             bit_cnt <= bit_cnt - 1'b1;
75             if (bit_cnt == 1) state <= FINISH;
76         end
77     end
78     FINISH: begin
79         rx_data <= shift_rx;
80         cs_n <= 1'b1;
81         done <= 1'b1;
82         busy <= 1'b0;
83         state <= IDLE;
84     end
85 endcase
86 end
87 end
88 endmodule
89

```

Pro Tips

- Know all four SPI modes (CPOL/CPHA combinations) and which edge is used to sample/drive data for each.
- In CPOL=1 CPHA=1 (Mode 3), the clock idle is high and data is sampled on the rising edge.
- For high-speed SPI, the MISO setup/hold time relative to SCLK is the critical path. Add output delay constraints in your SDC file.

Follow-Up Questions

1. Implement SPI Mode 3 (CPOL=1, CPHA=1).
2. Design an SPI Slave to complement this master.
3. How do you handle multi-slave SPI with multiple CS_N lines?
4. What is the maximum SPI clock frequency limited by?
5. How does SPI differ from QSPI (Quad SPI)?

Question 19

Design a Two-Flop CDC Synchronizer

INTERMEDIATE

Companies That Ask This Question

NVIDIA, AMD, Intel, Qualcomm, Apple, Broadcom, ARM

Problem Statement

Design a two-stage flip-flop synchronizer for crossing a single-bit control signal between two unrelated clock domains. Explain metastability, MTBF, and why exactly two stages are used (not one or three).

Verilog Solution - cdc_sync.v

```

1      // -----
2      // Module   : cdc_sync2
3      // Purpose  : 2-Flop CDC Synchronizer (single-bit signal)
4      // IMPORTANT: Both FFs must be placed in the same clock
5      //              domain and close together (timing constraint).
6      // -----
7      module cdc_sync2 #(
8          parameter RESET_VAL = 1'b0
9      )(
10         input  wire dst_clk,    // Destination clock domain
11         input  wire dst_rst,    // Destination domain reset
12         input  wire src_signal, // Asynchronous input (from src domain)
13         output wire dst_signal // Synchronized output
14     );
15     (* ASYNC_REG = "TRUE" *) // Xilinx attribute: no optimization
16     reg sync_ff1, sync_ff2;
17     always @(posedge dst_clk) begin
18         if (dst_rst) begin
19             sync_ff1 <= RESET_VAL;
20             sync_ff2 <= RESET_VAL;
21         end else begin
22
23         end
24         end
25         assign dst_signal = sync_ff2;
26     endmodule
27

```

RTL Design Explanation:

Metastability Explained

When a signal violates setup/hold time of a flip-flop, the FF output enters a metastable state — neither 0 nor 1. Given enough time (the resolution time), it will settle. The two-flop synchronizer gives the first FF one full clock period to resolve before the second FF samples it. **MTBF** (Mean Time Between Failure) is exponential in the resolution time: $MTBF \propto e^{T_{res}/\tau}$, where T_{res} is the clock period minus FF delays. Three flops give better MTBF but add latency. For most designs, two flops give $MTBF > 10^{12}$ years at 1 GHz — which is why the industry

standard is **two flops**.

Common Interview Mistakes

- **Using one flop:** Insufficient time for metastability resolution.
- **Synchronizing buses:** A two-flop synchronizer is only valid for single-bit signals. For multi-bit buses, use a Gray-coded counter or a handshake protocol.
- **Missing timing constraints:** Without `set_false_path` or `set_max_delay -datapath_only` in SDC, the tools may try to time the path across domains, which is incorrect.

Interviewer's Perspective

Why asked: CDC is one of the leading causes of silicon bugs. Companies test this heavily.

Evaluating: Metastability understanding, MTBF, why 2 flops, multi-bit CDC awareness, SDC constraints.

Impress: Immediately say “Two flops for single-bit signals only — multi-bit buses need Gray encoding or handshake.”

Follow-Up Questions

1. How do you synchronize a multi-bit bus across clock domains?
2. Explain Gray encoding and why it solves multi-bit CDC.
3. What SDC constraint do you apply to the CDC path?
4. When would you use 3 flops instead of 2?
5. How does an asynchronous FIFO solve the CDC problem for data streaming?

Question 20

Design a Round-Robin Arbiter

INTERMEDIATE

Companies That Ask This Question

NVIDIA, AMD, Intel, Qualcomm, Broadcom, ARM, Marvell

Problem Statement

Design a 4-client round-robin arbiter. Each client raises a req line. The arbiter grants one client at a time (gnt). After each grant, priority rotates to the next client. The arbiter must be starvation-free.

Verilog Solution - rr_arbiter.v

```

1      // -----
2      // Module : rr_arbiter
3      // Purpose : 4-client Round-Robin Arbiter
4      // -----
5      module rr_arbiter #(
6          parameter N = 4
7      )(
8          input wire      clk,
9          input wire      rst,
10         input wire [N-1:0] req,    // Client requests
11         output reg [N-1:0] gnt     // Grants (one-hot)
12     );
13     reg [N-1:0] priority_mask; // Rotating priority pointer
14     // Double the request vector for rotation trick
15     wire [2*N-1:0] req_double = {req, req};
16     wire [2*N-1:0] gnt_double;
17     wire [N-1:0] gnt_raw;
18     // Masked priority encoder: find first set starting from mask
19     wire [2*N-1:0] masked_req = req_double & ~({2*N{1'b1}} <<
20         $clog2(N)'(priority_mask));
21     // Simple find-first-set in 2N space
22     genvar i;
23     generate
24     for (i = 0; i < 2*N; i = i + 1) begin : gnt_gen
25         if (i == 0)
26             assign gnt_double[i] = masked_req[i];
27         else
28             assign gnt_double[i] = masked_req[i] & ~(|masked_req[i-1:0]);
29         end
30     endgenerate
31     assign gnt_raw = gnt_double[N-1:0] | gnt_double[2*N-1:N];
32     always @(posedge clk) begin
33         if (rst) begin
34             gnt      <= {N{1'b0}};
35             priority_mask <= {N{1'b1}}; // All bits set = no masking
36         end else begin
37             if (!req) begin
38                 gnt <= gnt_raw;
39                 // Next priority: rotate past granted client
40                 priority_mask <= ~((gnt_raw - 1'b1) | gnt_raw);
41             end else begin

```



```
42     gnt <= {N{1'b0}};
43     end
44     end
45     end
46     endmodule
47
```

Pro Tips

The **double-vector rotation trick** is the standard industry method for implementing round-robin arbitration: by doubling the request vector and starting the priority search from the current mask position, you get correct rotation without needing complex modular arithmetic. This technique scales to any power-of-2 N.

Follow-Up Questions

1. How does round-robin differ from weighted round-robin (WRR)?
2. Implement a weighted arbiter where Client 0 gets 2x bandwidth.
3. What is starvation, and how does round-robin prevent it?
4. How do you implement a 16-client round-robin in a tree structure?
5. What is a two-level arbiter, and where is it used?

Question 21

Design a PWM Generator

INTERMEDIATE

Companies That Ask This Question

Texas Instruments, Intel, Qualcomm, Samsung, MediaTek

Problem Statement

Design a PWM (Pulse Width Modulation) generator with parameterizable resolution. The duty cycle should be software-configurable via a register input. Include a period-complete flag.

Verilog Solution - pwm_gen.v

```

1      // -----
2      // Module : pwm_gen
3      // Purpose : PWM Generator with Configurable Duty Cycle
4      // Resolution: BITS-bit counter -> 2^BITS steps
5      // -----
6      module pwm_gen #(
7          parameter BITS = 8    // Resolution: 256 steps
8      )(
9          input wire      clk,
10         input wire      rst,
11         input wire [BITS-1:0] duty,    // 0=0%, 255=~100%
12         output reg      pwm_out,
13         output wire      period_done // Pulses at period start
14     );
15     reg [BITS-1:0] counter;
16     // Counter wraps freely
17     always @(posedge clk) begin
18         if (rst) counter <= {BITS{1'b0}};
19         else counter <= counter + 1'b1;
20     end
21     // Compare: output high when counter < duty
22     always @(posedge clk) begin
23         if (rst) pwm_out <= 1'b0;
24         else pwm_out <= (counter < duty);
25     end
26     // Period complete: fires when counter wraps to 0
27     assign period_done = (counter == {BITS{1'b0}});
28     endmodule
29

```

Follow-Up Questions

1. How do you synchronize **duty** updates to avoid glitches?
2. Implement a center-aligned PWM (triangle waveform for motor control).
3. What is dead-time insertion in a half-bridge PWM driver?
4. How is PWM used for LED brightness control?
5. Design a sigma-delta DAC using PWM + low-pass filter.

Question 22

Design a Johnson Counter

INTERMEDIATE

Companies That Ask This Question

Intel, Texas Instruments, Samsung, Marvell

Problem Statement

Design a parameterized Johnson counter (twisted ring counter). The counter cycles through $2 \times N$ unique states. Include a state-to-decoded output (one-hot style decoding).

Verilog Solution - johnson_counter.v

```

1      // -----
2      // Module : johnson_counter
3      // Purpose : Parameterized Johnson Counter
4      // States : 2*N unique one-hot-like states
5      // -----
6      module johnson_counter #(
7          parameter N = 4
8      )(
9          input wire      clk,
10         input wire      rst,
11         output reg [N-1:0] q
12     );
13     // Feedback: invert MSB and feed into LSB
14     always @(posedge clk) begin
15         if (rst) q <= {N{1'b0}};
16         else    q <= {~q[0], q[N-1:1]}; // Right shift, ~LSB -> MSB
17     end
18     endmodule
19 
```

RTL Design Explanation: The Johnson counter shifts right and feeds the inverted LSB into the MSB. For $N=4$, it cycles through 8 states: $0000 \rightarrow 1000 \rightarrow 1100 \rightarrow 1110 \rightarrow 1111 \rightarrow 0111 \rightarrow 0011 \rightarrow 0001 \rightarrow 0000$.

Pro Tips

Johnson counters are used in phase-shifted clock generation, FSM one-hot encoding alternatives, and glitch-free frequency dividers. The $2N$ states with only one bit changing per transition makes them naturally glitch-safe.

Follow-Up Questions

1. How does a Johnson counter differ from a ring counter?
2. Implement a ring counter (circular shift, no inversion).
3. What are the valid states of a 4-bit Johnson counter?
4. How do you detect lockup states in a Johnson counter?
5. Design a 3-phase clock generator using a 6-stage Johnson counter.

Question 23

Design a Gray Code Counter

INTERMEDIATE

Companies That Ask This Question

NVIDIA, AMD, Intel, Qualcomm, ARM

Problem Statement

Design a parameterized Gray code counter. Include binary-to-Gray and Gray-to-binary converters as sub-modules. Explain why Gray counters are used in async FIFO designs.

Verilog Solution - gray_counter.v

```

1      //-----
2      // Module : bin2gray / gray2bin / gray_counter
3      // Purpose : Gray Code Utilities
4      //-----
5      module bin2gray #(parameter N = 4)(
6      input wire [N-1:0] bin,
7      output wire [N-1:0] gray
8      );
9      assign gray = bin ^ (bin >> 1); // Standard XOR formula
10     endmodule
11     module gray2bin #(parameter N = 4)(
12     input wire [N-1:0] gray,
13     output reg [N-1:0] bin
14     );
15     integer i;
16     always @(*) begin
17     bin[N-1] = gray[N-1]; // MSB passes through
18     for (i = N-2; i >= 0; i = i - 1)
19     bin[i] = bin[i+1] ^ gray[i]; // Cascade XOR
20     end
21     endmodule
22     //-----
23     module gray_counter #(parameter N = 4)(
24     input wire clk,
25     input wire rst,
26     output wire [N-1:0] gray_out
27     );
28     reg [N-1:0] bin_count;
29     // Binary counter underneath
30     always @(posedge clk) begin
31     if (rst) bin_count <= {N{1'b0}};
32     else bin_count <= bin_count + 1'b1;
33     end
34     // Convert to Gray on the fly
35     assign gray_out = bin_count ^ (bin_count >> 1);
36     endmodule
37

```

Interviewer's Perspective

Why asked: Gray codes are critical for asynchronous FIFO pointer synchronization. Understanding them shows CDC maturity.

Evaluating: Binary-to-Gray formula, Gray-to-Binary cascade XOR, why Gray eliminates multi-bit transitions.

Impress: “Gray code changes only one bit per count, so even if the synchronized value is one count off, the FIFO never misreads full/empty status as wildly incorrect.”

Follow-Up Questions

1. Why does an async FIFO use Gray-coded pointers?
2. What happens if you use binary pointers in an async FIFO?
3. How many bits change in a 4-bit Gray code between adjacent counts?
4. Convert the binary number 1011 to Gray code by hand.
5. Implement a Gray code ROM using a lookup table.

Question 24

Design a CRC-8 Generator

INTERMEDIATE

Companies That Ask This Question

Intel, Qualcomm, Broadcom, Marvell, Samsung

Problem Statement

Design a CRC-8 generator using the polynomial $x^8 + x^2 + x + 1$ (0x07). Process one byte per clock cycle. Output the 8-bit CRC after all data bytes have been processed.

Verilog Solution - crc8.v

```

1      // -----
2      // Module   : crc8
3      // Purpose  : CRC-8 Generator (Poly: 0x07 =  $x^8 + x^2 + x + 1$ )
4      // -----
5      module crc8 (
6          input wire      clk,
7          input wire      rst,
8          input wire      valid,    // Input byte is valid
9          input wire [7:0] data_in,
10         output reg [7:0] crc_out
11     );
12     reg [7:0] crc_reg;
13     wire [7:0] crc_next;
14     // Combinational CRC update (serial XOR through polynomial)
15     function [7:0] crc8_update;
16     input [7:0] crc;
17     input [7:0] data;
18     integer i;
19     reg [7:0] tmp;
20     begin
21         tmp = crc ^ data;
22         for (i = 0; i < 8; i = i + 1) begin
23             if (tmp[7])
24                 tmp = (tmp << 1) ^ 8'h07; // XOR with polynomial
25             else
26                 tmp = tmp << 1;
27         end
28         crc8_update = tmp;
29     end
30     endfunction
31     assign crc_next = crc8_update(crc_reg, data_in);
32     always @(posedge clk) begin
33         if (rst)      crc_reg <= 8'hFF; // Init value
34         else if (valid) crc_reg <= crc_next;
35     end
36     assign crc_out = crc_reg;
37 endmodule
38

```

Pro Tips

In production RTL, CRC is often implemented as a **parallel combinational network** (not the serial loop shown above). The parallel implementation unrolls the loop and maps each output bit to a XOR tree of input bits. Tools like `crcany` can generate the parallel XOR equations automatically.

Follow-Up Questions

1. Implement a CRC-32 using the Ethernet polynomial (0x04C11DB7).
2. What is the difference between CRC and parity?
3. How do you implement CRC in a parallel (unrolled) architecture?
4. What types of errors can CRC-8 detect but not correct?
5. How do you verify a received packet using its CRC?

Question 25

Design a Parameterized Pipeline Register Stage

INTERMEDIATE

Companies That Ask This Question

NVIDIA, AMD, Intel, Qualcomm, Apple, ARM

Problem Statement

Design a generic pipeline register stage with configurable depth (number of stages). Include a valid bit pipeline and a bubble/flush mechanism. This should serve as a retiming-friendly pipeline primitive.

Verilog Solution - pipe_stage.v

```

1      // -----
2      // Module : pipe_stage
3      // Purpose : Parameterized Pipeline Register with Valid/Flush
4      // -----
5      module pipe_stage #(
6          parameter DATA_WIDTH = 32,
7          parameter STAGES      = 3
8      )(
9          input  wire          clk,
10         input  wire          rst,
11         input  wire          flush,    // Kill pipeline
12         input  wire          stall,    // Pause pipeline
13         input  wire          din_valid,
14         input  wire [DATA_WIDTH-1:0] din,
15         output wire          dout_valid,
16         output wire [DATA_WIDTH-1:0] dout
17     );
18     reg [DATA_WIDTH-1:0] data_pipe [0:STAGES-1];
19     reg                  valid_pipe [0:STAGES-1];
20     integer i;
21     always @(posedge clk) begin
22         if (rst || flush) begin
23             for (i = 0; i < STAGES; i = i + 1) begin
24                 data_pipe[i] <= {DATA_WIDTH{1'b0}};
25                 valid_pipe[i] <= 1'b0;
26             end
27         end else if (!stall) begin
28             data_pipe[0] <= din;
29             valid_pipe[0] <= din_valid;
30             for (i = 1; i < STAGES; i = i + 1) begin
31                 data_pipe[i] <= data_pipe[i-1];
32                 valid_pipe[i] <= valid_pipe[i-1];
33             end
34         end
35     end
36     assign dout          = data_pipe[STAGES-1];
37     assign dout_valid = valid_pipe[STAGES-1];
38 endmodule
39

```


Common Interview Mistakes

- **Forgetting the valid bit pipeline:** Data registers alone are not sufficient. The valid bit must be pipelined in lockstep.
- **Flush vs Reset:** Flush clears the pipeline mid-operation (branch misprediction, exception). Reset is power-on initialization. Treat them differently if flush must be fast-path and reset can be slow.
- **Stall and flush simultaneously:** Stall should take priority over flush in some designs. Clarify the priority during the interview.

Follow-Up Questions

1. How do you add forwarding (bypass) logic across pipeline stages?
2. Implement a 5-stage RISC pipeline (IF, ID, EX, MEM, WB) at a high level.
3. What is retiming, and how do synthesizers use pipeline registers for it?
4. How does a stall propagate backward through a pipeline?
5. What is the throughput penalty of a 5-stage pipeline with 1-cycle stall per instruction?

Question 26

Design a Vending Machine FSM

INTERMEDIATE

Companies That Ask This Question

Texas Instruments, Qualcomm, Samsung, Synopsys, Intel

Problem Statement

Design a Mealy FSM for a vending machine. The machine sells items worth 15 cents. It accepts nickels (5¢) and dimes (10¢). When enough money is inserted, it dispenses the item and returns change if any.

Verilog Solution - vending_machine.v

```

1      // -----
2      // Module : vending_machine
3      // Purpose : Vending Machine FSM (Mealy) - item costs 15c
4      // nickel=1, dime=2 (one-hot coin input)
5      // -----
6      module vending_machine (
7          input wire clk,
8          input wire rst,
9          input wire nickel,    // 5-cent coin inserted
10         input wire dime,      // 10-cent coin inserted
11         output reg dispense,   // Item dispensed
12         output reg ret_nickel // Return 5 cents change
13     );
14     // States represent credit accumulated: 0,5,10 cents
15     typedef enum reg [1:0] {
16         S0 = 2'b00,    // 0 cents
17         S5 = 2'b01,    // 5 cents
18         S10 = 2'b10    // 10 cents
19     } state_t;
20     state_t state, next_state;
21     // State register
22     always @(posedge clk) begin
23         if (rst) state <= S0;
24         else state <= next_state;
25     end
26     // Next-state and output logic (Mealy)
27     always @(*) begin
28         next_state = state;
29         dispense = 1'b0;
30         ret_nickel = 1'b0;
31         case (state)
32             S0: begin
33                 if (nickel) next_state = S5;
34                 else if (dime) next_state = S10;
35             end
36             S5: begin
37                 if (nickel) next_state = S10;
38                 else if (dime) begin
39                     dispense = 1'b1; // Got 15c, dispense!
40                     next_state = S0;

```

```
41     end
42     end
43     S10: begin
44         if (nickel) begin
45             dispense = 1'b1;
46             next_state = S0;
47         end
48         else if (dime) begin
49             dispense = 1'b1; // Got 20c: dispense + change
50             ret_nickel = 1'b1;
51             next_state = S0;
52         end
53     end
54     default: next_state = S0;
55 endcase
56 end
57 endmodule
58
```

Follow-Up Questions

1. Add a quarter (25¢) coin input.
2. Change item price to 20¢ — how do states change?
3. Convert this Mealy FSM to a Moore FSM.
4. Add a cancel/refund button.
5. How would you implement this for multiple item prices?

Question 27

Design a Debouncer Circuit

INTERMEDIATE

Companies That Ask This Question

Texas Instruments, Intel, Samsung, Marvell, MediaTek

Problem Statement

Design a button debouncer that removes glitches from a mechanical button press. The output should only change state after the input has been stable for a programmable number of clock cycles (e.g., 20ms at a given clock frequency).

Verilog Solution - debouncer.v

```

1      // -----
2      // Module : debouncer
3      // Purpose : Button Debouncer (counter-based)
4      // -----
5      module debouncer #(
6          parameter CLK_FREQ = 50_000_000, // 50 MHz
7          parameter DEBOUNCE_MS = 20      // 20 ms debounce window
8      )(
9          input wire clk,
10         input wire rst,
11         input wire btn_raw, // Raw (bouncing) button input
12         output reg btn_stable, // Debounced output
13         output wire btn_press, // Single-cycle pulse on press
14         output wire btn_release // Single-cycle pulse on release
15     );
16     localparam DEBOUNCE_COUNT = (CLK_FREQ / 1000) * DEBOUNCE_MS;
17     localparam CTR_BITS = $clog2(DEBOUNCE_COUNT + 1);
18     reg [CTR_BITS-1:0] counter;
19     reg sync1, sync2; // 2-flop synchronizer for btn
20     // Synchronize button to clock domain
21     always @(posedge clk) begin
22         sync1 <= btn_raw;
23         sync2 <= sync1;
24     end
25     // Counter resets when input changes; output updates when stable
26     always @(posedge clk) begin
27         if (rst) begin
28             counter <= 0;
29             btn_stable <= 1'b0;
30         end else begin
31             if (sync2 != btn_stable) begin
32                 if (counter == DEBOUNCE_COUNT - 1) begin
33                     btn_stable <= sync2;
34                     counter <= 0;
35                 end else begin
36                     counter <= counter + 1'b1;
37                 end
38             end else begin
39                 counter <= 0;
40             end
41         end
42     end

```

```
41     end
42     end
43     // Edge detection for pulse outputs
44     reg btn_stable_d;
45     always @(posedge clk) btn_stable_d <= btn_stable;
46     assign btn_press    = btn_stable & ~btn_stable_d;
47     assign btn_release = ~btn_stable & btn_stable_d;
48     endmodule
49
```

Follow-Up Questions

1. Why is the 2-flop synchronizer necessary here?
2. Implement debouncing using a shift register instead of a counter.
3. What is the maximum bounce duration for typical mechanical buttons?
4. How do you test a debouncer in simulation?
5. Add a long-press detection (>1 second held = long press).

Question 28

Design an Up/Down Counter with Load

INTERMEDIATE

Companies That Ask This Question

Texas Instruments, Intel, Samsung, Synopsys

Problem Statement

Design a parameterized up/down counter with: synchronous load, count direction control, borrow and carry outputs, and terminal count in both directions.

Verilog Solution - updown_counter.v

```

1      // -----
2      // Module   : updown_counter
3      // Purpose  : Parameterized Up/Down Counter with Load
4      // -----
5      module updown_counter #(
6      parameter WIDTH = 8
7      )(
8      input  wire      clk,
9      input  wire      rst,
10     input  wire      en,        // Count enable
11     input  wire      up_dn,    // 1=up, 0=down
12     input  wire      load,     // Synchronous load
13     input  wire [WIDTH-1:0] load_val,
14     output reg  [WIDTH-1:0] count,
15     output wire      carry,    // Overflow in up direction
16     output wire      borrow   // Underflow in down direction
17 );
18     localparam MAX_VAL = {WIDTH{1'b1}};
19     assign carry = (count == MAX_VAL) & en & up_dn;
20     assign borrow = (count == {WIDTH{1'b0}}) & en & ~up_dn;
21     always @(posedge clk) begin
22         if (rst) count <= {WIDTH{1'b0}};
23         else if (load) count <= load_val;
24         else if (en) begin
25             if (up_dn) count <= count + 1'b1;
26             else count <= count - 1'b1;
27         end
28     end
29 endmodule
30

```

Follow-Up Questions

1. Design a BCD up/down counter (0 to 9).
2. Implement a countdown timer using this counter.
3. How does this differ from a loadable modulo-N counter?
4. Add a rollover/clamp option (saturate at min/max vs wrap).
5. How would you cascade two 8-bit counters to make a 16-bit counter?

Question 29

Design a Pulse Stretcher / Monostable Multivibrator

INTERMEDIATE

Companies That Ask This Question

Intel, Texas Instruments, Samsung, Synopsys, ARM

Problem Statement

Design a pulse stretcher: a short input pulse (possibly 1 clock cycle) should be extended to a programmable duration in clock cycles. Useful for interrupt lines and status flag generation.

Verilog Solution - pulse_stretch.v

```

1      // -----
2      // Module : pulse_stretch
3      // Purpose : Extend pulse to N clock cycles duration
4      // -----
5      module pulse_stretch #(
6          parameter PULSE_LEN = 16    // Output pulse width in cycles
7      )(
8          input  wire clk,
9          input  wire rst,
10         input  wire trigger,        // Input pulse (may be 1 cycle)
11         output reg stretched        // Extended output
12     );
13     localparam CTR_W = $clog2(PULSE_LEN + 1);
14     reg [CTR_W-1:0] count;
15     always @(posedge clk) begin
16         if (rst) begin
17             count    <= 0;
18             stretched <= 1'b0;
19         end else begin
20             if (trigger) begin        // Re-trigger on any pulse
21                 count    <= PULSE_LEN[CTR_W-1:0];
22                 stretched <= 1'b1;
23             end else if (count > 0) begin
24                 count    <= count - 1'b1;
25                 stretched <= 1'b1;
26             end else begin
27                 stretched <= 1'b0;
28             end
29         end
30     end
31 endmodule
32

```

Follow-Up Questions

1. Design a pulse compressor (narrow a wide pulse to 1 cycle).
2. Implement a retriggerable monostable that resets the timer on each new pulse.
3. How do you implement a non-retriggerable version?
4. Design an edge detector that generates a single-cycle pulse from a level signal.

5. Where is a pulse stretcher used in interrupt handling hardware?

Question 30

Design a Positive and Negative Edge Detector

INTERMEDIATE

Companies That Ask This Question

Intel, AMD, Texas Instruments, ARM, Synopsys

Problem Statement

Design an edge detector that generates single-cycle pulses for rising edge, falling edge, and any edge (either). The detector must be fully synchronous.

Verilog Solution - edge_detect.v

```

1      // -----
2      // Module : edge_detect
3      // Purpose : Synchronous Positive/Negative Edge Detector
4      // -----
5      module edge_detect (
6          input wire clk,
7          input wire rst,
8          input wire sig,          // Input signal (should be sync'd)
9          output wire pos_edge,    // Rising edge pulse
10         output wire neg_edge,    // Falling edge pulse
11         output wire any_edge     // Either edge pulse
12     );
13     reg sig_d; // Delayed version (1 cycle)
14     always @(posedge clk) begin
15         if (rst) sig_d <= 1'b0;
16         else    sig_d <= sig;
17     end
18     // Edge detection: current XOR delayed
19     assign pos_edge = sig & ~sig_d; // 0->1 transition
20     assign neg_edge = ~sig & sig_d; // 1->0 transition
21     assign any_edge = sig ^ sig_d;  // Either transition
22 endmodule
23

```

Pro Tips

This pattern (`sig & ~sig_d`) is one of the most frequently used primitives in RTL design. It appears in interrupt edge detection, protocol start-of-frame detection, clock edge events, and button press logic. **Memorize this idiom.**

Common Interview Mistakes

- **Applying edge detect to async signals:** Always synchronize the input with a 2-flop synchronizer before edge detecting.
- **Using `posedge sig` in `always` block:** This creates a clock (using a data signal as a clock), which is a DRC violation in most flows.

Follow-Up Questions

1. How do you detect both edges without duplicating logic?
2. Design a “toggle on rising edge” circuit using this detector.
3. What happens if `sig` is an asynchronous input?
4. Implement a frequency meter using edge detection and a counter.
5. Where is edge detection used in I2C START/STOP condition detection?

Expert Questions

Chapter Overview

These five questions are asked at senior/staff engineer and principal engineer levels at top-tier semiconductor companies. They require deep microarchitecture knowledge, low-power expertise, formal reasoning about correctness, and production-quality RTL craftsmanship.

Question 46

Design a Parameterized Carry-Lookahead Adder

EXPERT

Companies That Ask This Question

Intel, AMD, Apple, ARM, NVIDIA, Qualcomm

Problem Statement

Implement a 32-bit Carry-Lookahead Adder (CLA) in structural Verilog. Build it from 4-bit CLA cells in a hierarchical manner. Explain the generate (G) and propagate (P) logic. Compare timing with a ripple-carry adder. Derive the equations from first principles during the interview.

Interview Requirements: Derive G , P , and carry equations from Boolean algebra before writing any code. Draw the 4-bit cell block diagram.

Verilog Solution - cla_adder.v

```

1      // -----
2      // Module   : cla_4bit / cla_32bit
3      // Purpose  : Carry-Lookahead Adder (4-bit cell + 32-bit top)
4      //
5      // THEORY:
6      //   Gi = Ai & Bi           (Generate: this bit WILL produce carry)
7      //   Pi = Ai ^ Bi           (Propagate: this bit WILL pass a carry)
8      //   Ci+1 = Gi + Pi*Ci      (Carry at each bit position)
9      //
10     // For 4-bit group:
11     //   C1 = G0 + P0*C0
12     //   C2 = G1 + P1*G0 + P1*P0*C0
13     //   C3 = G2 + P2*G1 + P2*P1*G0 + P2*P1*P0*C0
14     //   C4 = G3 + P3*G2 + P3*P2*G1 + P3*P2*P1*G0 + P3*P2*P1*P0*C0
15     //   GG = G3 + P3*G2 + P3*P2*G1 + P3*P2*P1*G0      (Group Generate)
16     //   GP = P3*P2*P1*P0                                (Group Propagate)
17     // -----
18     module cla_4bit (
19     input wire [3:0] a, b,
20     input wire      cin,
21     output wire [3:0] sum,
22     output wire      cout,
23     output wire      G_grp, // Group generate (for hierarchical CLA)
24     output wire      P_grp  // Group propagate
25     );
26     wire [3:0] G, P; // Bit-level generate/propagate
27     wire [3:0] C;    // Internal carries (C[0]=cin, C[1..3]=computed)
28     assign G = a & b;
29     assign P = a ^ b;
30     // CLA carry equations (parallel, not ripple!)
31     assign C[0] = cin;
32     assign C[1] = G[0] | (P[0] & C[0]);
33     assign C[2] = G[1] | (P[1] & G[0]) | (P[1] & P[0] & C[0]);
34     assign C[3] = G[2] | (P[2] & G[1]) | (P[2] & P[1] & G[0]) |
35     (P[2] & P[1] & P[0] & C[0]);
36     // Sum

```

```

37     assign sum = P ^ {C[3], C[2], C[1], C[0]};
38     // Carry out
39     assign cout = G[3] | (P[3] & G[2]) | (P[3] & P[2] & G[1]) |
40     (P[3] & P[2] & P[1] & G[0]) |
41     (P[3] & P[2] & P[1] & P[0] & C[0]);
42     // Group G/P for hierarchical composition
43     assign G_grp = G[3] | (P[3] & G[2]) | (P[3] & P[2] & G[1]) |
44     (P[3] & P[2] & P[1] & G[0]);
45     assign P_grp = P[3] & P[2] & P[1] & P[0];
46     endmodule
47     // -----
48     // 32-bit CLA: 8 x 4-bit cells with group CLA logic
49     // -----
50     module cla_32bit (
51     input wire [31:0] a, b,
52     input wire      cin,
53     output wire [31:0] sum,
54     output wire      cout
55     );
56     wire [7:0] G_grp, P_grp; // Group G/P from each 4-bit cell
57     wire [8:0] C_grp;        // Group carries between 4-bit cells
58     assign C_grp[0] = cin;
59     // Group-level carry lookahead (same formula, applied to groups)
60     assign C_grp[1] = G_grp[0] | (P_grp[0] & C_grp[0]);
61     assign C_grp[2] = G_grp[1] | (P_grp[1] & G_grp[0]) |
62     (P_grp[1] & P_grp[0] & C_grp[0]);
63     assign C_grp[3] = G_grp[2] | (P_grp[2] & G_grp[1]) |
64     (P_grp[2] & P_grp[1] & G_grp[0]) |
65     (P_grp[2] & P_grp[1] & P_grp[0] & C_grp[0]);
66
67     genvar i;
68     generate
69     for (i = 0; i < 8; i = i + 1) begin : cla_cells
70     cla_4bit u_cla (
71     .a    (a[4*i+3 : 4*i]),
72     .b    (b[4*i+3 : 4*i]),
73     .cin  (C_grp[i]),
74     .sum  (sum[4*i+3 : 4*i]),
75     .G_grp(G_grp[i]),
76     .P_grp(P_grp[i]),
77
78     );
79     end
80     endgenerate
81     assign cout = C_grp[8];
82     endmodule
83

```

Timing Discussion:

Adder Type	Delay (gates)	Area	32-bit Example
Ripple-Carry (RCA)	$O(N)$	Low	32 gate delays
Carry-Lookahead (CLA)	$O(\log N)$	Medium	5 gate delays
Carry-Select (CSLA)	$O(\sqrt{N})$	High	8 gate delays
Prefix (Kogge-Stone)	$O(\log N)$	Highest	4 gate delays

Common Interview Mistakes

- **Confusing G and P:** Generate is AND ($G_i = A_i \& B_i$). Propagate is XOR ($P_i = A_i \oplus B_i$). Note: some texts define P as OR (for carry purposes) — clarify which convention you are using.
- **Ripple-carry the group carries:** Building 4-bit CLA cells but then cascading their carryouts in a ripple chain defeats the entire purpose. Apply the lookahead logic at the group level too.
- **Forgetting the generate-for sum:** The sum is $P \oplus \{C[3], C[2], C[1], C[0]\}$, not $A \oplus B \oplus C$ naively.

Pro Tips

- Modern synthesizers (Synopsys DC, Cadence Genus) automatically map adder logic to the fastest available adder structures in the standard cell library. In practice, you write $a + b$ in RTL and the tool picks the optimal adder.
- However, **knowing the theory** of adder architectures (CLA, Kogge-Stone, Brent-Kung, Han-Carlson) is expected at senior level — especially when discussing timing closure or custom datapath design.
- For FPGA: adder logic maps to the dedicated carry chain (CARRY4/CARRY8 primitives), which is a fast, dedicated ripple-carry structure optimized in silicon.

Interviewer's Perspective

Why asked: At principal engineer level, understanding arithmetic unit implementation distinguishes true digital design experts from Verilog coders.

Evaluating: Derivation of G/P equations from first principles, hierarchical CLA structure, timing analysis.

Impress: Mention prefix adder trees (Kogge-Stone, Brent-Kung) and compare their area-delay tradeoffs. Reference Weste & Harris CMOS VLSI Design.

Follow-Up Questions

1. Derive the Kogge-Stone prefix tree structure for a 16-bit adder.
2. Compare Brent-Kung and Kogge-Stone in terms of area and wire count.
3. Implement a carry-select adder as an alternative to CLA.
4. How does the FPGA carry chain achieve low-latency addition?
5. What is the Booth encoding, and why is it used in multiplier design?

Practice Challenge

Derive the 4-bit CLA carry equations C_1 through C_4 completely from first principles on paper. Then extend to 8-bit and identify the pattern. Implement a generalized N -bit CLA using a `generate` loop.

Question 47

Design a Two-Stage Out-of-Order Issue Queue

EXPERT

Companies That Ask This Question

NVIDIA, AMD, Intel, Apple, ARM, Qualcomm

Problem Statement

Design a simplified issue queue for an out-of-order processor. The queue holds 4 entries. Each entry contains: valid bit, opcode (4-bit), src register indices (5-bit each), ready bits for src1 and src2, and destination register index. A wakeup broadcast updates ready bits when a result is written. Issue the oldest ready instruction (in-order issue among ready instructions).

Verilog Solution - issue_queue.v

```

1  // -----
2  // Module : issue_queue
3  // Purpose : Simplified 4-entry Out-of-Order Issue Queue
4  // Wakeup + Select logic for DoO execution
5  // -----
6  module issue_queue #(
7      parameter ENTRIES = 4,
8      parameter REG_W   = 5,
9      parameter OPC_W   = 4
10 ) (
11     input wire      clk,
12     input wire      rst,
13     // Dispatch (write new instruction into queue)
14     input wire      dispatch_valid,
15     input wire [OPC_W-1:0] dispatch_opc,
16     input wire [REG_W-1:0] dispatch_src1,
17     input wire [REG_W-1:0] dispatch_src2,
18     input wire [REG_W-1:0] dispatch_dst,
19     input wire      dispatch_src1_rdy, // Already ready?
20     input wire      dispatch_src2_rdy,
21     // Wakeup broadcast (from execution units)
22     input wire      wakeup_valid,
23     input wire [REG_W-1:0] wakeup_tag, // Physical reg being written
24     // Issue output (to execution unit)
25     output reg      issue_valid,
26     output reg [OPC_W-1:0] issue_opc,
27     output reg [REG_W-1:0] issue_dst
28 );
29 // Queue storage
30 reg [OPC_W-1:0] q_opc  [0:ENTRIES-1];
31 reg [REG_W-1:0] q_src1 [0:ENTRIES-1];
32 reg [REG_W-1:0] q_src2 [0:ENTRIES-1];
33 reg [REG_W-1:0] q_dst  [0:ENTRIES-1];
34 reg            q_valid[0:ENTRIES-1];
35 reg            q_rdy1 [0:ENTRIES-1];
36 reg            q_rdy2 [0:ENTRIES-1];
37 reg [1:0]      q_age  [0:ENTRIES-1]; // Age for priority
38 integer i, j;
39 reg [ENTRIES-1:0] ready_mask; // Which entries are fully ready

```

```

40     reg [1:0]      select_entry; // Oldest ready entry index
41     reg           found;
42     // Wakeup: update ready bits on broadcast
43     always @(posedge clk) begin
44         if (rst) begin
45             for (i = 0; i < ENTRIES; i = i + 1) begin
46                 q_valid[i] <= 1'b0;
47                 q_rdy1[i]  <= 1'b0;
48                 q_rdy2[i]  <= 1'b0;
49                 q_age[i]   <= 2'b0;
50             end
51         end else begin
52             // Wakeup broadcast: set ready if source matches wakeup tag
53             for (i = 0; i < ENTRIES; i = i + 1) begin
54                 if (q_valid[i] && wakeup_valid) begin
55                     if (q_src1[i] == wakeup_tag) q_rdy1[i] <= 1'b1;
56                     if (q_src2[i] == wakeup_tag) q_rdy2[i] <= 1'b1;
57                 end
58             end
59             // Issue: find oldest ready entry
60             found      = 1'b0;
61             select_entry = 2'b0;
62             issue_valid <= 1'b0;
63             for (i = ENTRIES-1; i >= 0; i = i - 1) begin
64                 if (q_valid[i] && q_rdy1[i] && q_rdy2[i]) begin
65                     select_entry = i[1:0];
66                     found        = 1'b1;
67                 end
68             end
69             if (found) begin
70                 issue_valid <= 1'b1;
71                 issue_opc  <= q_opc[select_entry];
72                 issue_dst  <= q_dst[select_entry];
73                 q_valid[select_entry] <= 1'b0; // Deallocate
74             end
75             // Dispatch: find first free entry
76             if (dispatch_valid) begin
77                 for (i = 0; i < ENTRIES; i = i + 1) begin
78                     if (!q_valid[i] && !found) begin
79                         q_opc[i] <= dispatch_opc;
80                         q_src1[i] <= dispatch_src1;
81                         q_src2[i] <= dispatch_src2;
82                         q_dst[i]  <= dispatch_dst;
83                         q_rdy1[i] <= dispatch_src1_rdy;
84                         q_rdy2[i] <= dispatch_src2_rdy;
85                         q_valid[i] <= 1'b1;
86                     end
87                 end
88             end
89         end
90     end
91 endmodule
92

```

RTL Design Explanation:

- **Wakeup:** When an execution unit completes, it broadcasts the physical register tag. All issue queue entries compare their source tags against the wakeup tag and set their ready bits.
- **Select:** Among all ready entries, the oldest is chosen (age-based priority). In this simplified version, the lowest index in the ready mask is issued.

- **Dispatch:** New instructions are allocated to free entries on dispatch.
- **Issue:** The selected instruction is issued and the entry is freed.

Interviewer's Perspective

Why asked: CPUs designed at AMD, Intel, Apple, and ARM require engineers who understand the microarchitecture of superscalar OoO processors. This question separates architects from RTL engineers.

Evaluating: Wakeup-select pipeline, ready bit management, age-based priority, no structural hazards.

Impress: Explain the concept of a **CAM (Content-Addressable Memory)** for tag comparison and mention speculative wakeup.

Follow-Up Questions

1. What is speculative wakeup, and why can it cause pipeline squashes?
2. How does a Tomasulo algorithm differ from this simple issue queue?
3. Implement a scoreboard (in-order issue, out-of-order completion).
4. How does register renaming (RAT) prevent WAW and WAR hazards?
5. What is the critical path in the wakeup-select pipeline?

Question 48

Design a Low-Power Handshake Protocol (4-Phase)

EXPERT

Companies That Ask This Question

ARM, Intel, Qualcomm, Apple, NVIDIA, Broadcom

Problem Statement

Design a 4-phase handshake synchronizer for asynchronous CDC (as an alternative to FIFO-based CDC). The sender waits for acknowledgment before sending the next datum. Implement both the sender and receiver finite state machines. Explain why this is more robust than the 2-phase (toggle) method for bursty data.

Verilog Solution - handshake_cdc.v

```

1      // -----
2      // Modules : hs_sender / hs_receiver
3      // Purpose : 4-Phase Handshake CDC Synchronizer
4      // Protocol:
5      //   Phase 1: Sender asserts REQ (data stable)
6      //   Phase 2: Receiver samples data, asserts ACK
7      //   Phase 3: Sender de-asserts REQ
8      //   Phase 4: Receiver de-asserts ACK
9      // -----
10     module hs_sender #(parameter DATA_W = 8)(
11     input wire      src_clk,
12     input wire      src_rst,
13     input wire [DATA_W-1:0] send_data,
14     input wire      send_valid,    // Pulse to send
15     output reg      src_busy,      // Cannot accept new data
16     // Handshake signals (cross domain)
17     output reg      req,           // Request to receiver
18     output reg [DATA_W-1:0] data_out, // Data (stable during req)
19     input wire      ack_sync       // ACK synchronized to src_clk
20     );
21     localparam IDLE    = 2'd0;
22     localparam REQ_HI  = 2'd1; // REQ asserted, wait for ACK=1
23     localparam REQ_LO  = 2'd2; // REQ de-asserted, wait for ACK=0
24     reg [1:0] state;
25     always @(posedge src_clk or posedge src_rst) begin
26     if (src_rst) begin
27     state <= IDLE;
28     req <= 1'b0;
29     src_busy <= 1'b0;
30     end else begin
31     case (state)
32     IDLE: begin
33     src_busy <= 1'b0;
34     if (send_valid) begin
35     data_out <= send_data;
36     req <= 1'b1;
37     src_busy <= 1'b1;
38     state <= REQ_HI;
39     end

```

```

40     end
41     REQ_HI: begin
42     if (ack_sync) begin // Receiver acknowledged
43     req    <= 1'b0;
44     state <= REQ_LO;
45     end
46     end
47     REQ_LO: begin
48     if (!ack_sync) begin // Receiver de-asserted ACK
49     state    <= IDLE;
50     src_busy <= 1'b0;
51     end
52     end
53     default: state <= IDLE;
54     endcase
55     end
56     end
57     endmodule
58     // -----
59     module hs_receiver #(parameter DATA_W = 8)(
60     input wire      dst_clk,
61     input wire      dst_rst,
62     input wire      req_sync, // REQ synchronized to dst_clk
63     input wire [DATA_W-1:0] data_in, // Data from sender
64     output reg      ack, // Acknowledge
65     output reg [DATA_W-1:0] rx_data, // Received data (latched)
66     output reg      rx_valid // Pulse: data valid
67     );
68     localparam IDLE    = 2'd0;
69     localparam ACK_HI  = 2'd1;
70     localparam ACK_LO  = 2'd2;
71     reg [1:0] state;
72     always @(posedge dst_clk or posedge dst_rst) begin
73     if (dst_rst) begin
74     state    <= IDLE;
75     ack      <= 1'b0;
76     rx_valid <= 1'b0;
77     end else begin
78     rx_valid <= 1'b0;
79     case (state)
80     IDLE: begin
81     if (req_sync) begin // REQ detected
82     rx_data <= data_in; // Latch data
83     rx_valid <= 1'b1;
84     ack    <= 1'b1; // Send ACK
85     state  <= ACK_HI;
86     end
87     end
88     ACK_HI: begin
89     if (!req_sync) begin // REQ de-asserted
90     ack    <= 1'b0; // De-assert ACK
91     state <= ACK_LO;
92     end
93     end
94     ACK_LO: begin
95     if (!ack) state <= IDLE; // ACK fully cleared
96     end
97     default: state <= IDLE;
98     endcase
99     end
100    end
101    endmodule

```

102

4-Phase vs 2-Phase Handshake

- **2-Phase (toggle):** Simpler, one synchronizer. Works for single pulses. Cannot easily stream back-to-back data.
- **4-Phase (level):** Requires two synchronizers (REQ→dst, ACK→src). Adds 2–4 cycles latency per transfer but is unambiguous and robust for data transfer.
- **Throughput:** One transfer per 4-phase cycle = approx $4 \times T_{dst_clk}$ latency. For streaming data, use async FIFO instead.

Interviewer's Perspective

Why asked: Low-power and asynchronous design are growing fields. 4-phase handshake appears in clock domain crossings, power-gated domain interfaces (PMU handshakes), and asynchronous logic designs.

Evaluating: Protocol state machine, two-synchronizer necessity, latency analysis, comparison with 2-phase and FIFO approaches.

Impress: Mention GALS (Globally Asynchronous Locally Synchronous) architectures and where handshake protocols are used in Nokia/ARM chip designs.

Follow-Up Questions

1. What happens if the ACK synchronizer samples ACK at a metastable value?
2. How do you add a timeout to the sender in case the receiver never ACKs?
3. Derive the maximum throughput of the 4-phase handshake.
4. How does this compare with AXI valid-ready handshake latency?
5. Implement a 2-phase (toggle-based) handshake and compare code complexity.

Question 49

Design a Full AXI4 Lite Slave with Register Map

EXPERT

Companies That Ask This Question

ARM, NVIDIA, AMD, Intel, Qualcomm, Broadcom, Apple

Problem Statement

Implement a complete AXI4-Lite slave with a 4-register configuration map. Support both read and write channels (AW, W, B, AR, R). Handle all five AXI channels independently. Include BRESP and RRESP error responses for out-of-range addresses. This is the standard IP-level register block.

Verilog Solution - axi4lite_slave.v

```

1  // -----
2  // Module : axi4lite_slave
3  // Purpose : AXI4-Lite Slave Register Block (4 registers)
4  // -----
5  module axi4lite_slave #(
6      parameter DATA_W = 32,
7      parameter ADDR_W = 8
8  )(
9      input wire          aclk,
10     input wire          aresetn,
11     // Write Address Channel
12     input wire [ADDR_W-1:0] awaddr,
13     input wire          awvalid,
14     output reg          awready,
15     // Write Data Channel
16     input wire [DATA_W-1:0] wdata,
17     input wire [3:0]       wstrb,
18     input wire          wvalid,
19     output reg          wready,
20     // Write Response Channel
21     output reg [1:0]      bresp,    // 0=OKAY, 2=SLVERR
22     output reg          bvalid,
23     input wire          bready,
24     // Read Address Channel
25     input wire [ADDR_W-1:0] araddr,
26     input wire          arvalid,
27     output reg          arready,
28     // Read Data Channel
29     output reg [DATA_W-1:0] rdata,
30     output reg [1:0]      rresp,
31     output reg          rvalid,
32     input wire          rready
33 );
34 // Register bank
35 reg [DATA_W-1:0] regs [0:3];
36 reg [ADDR_W-1:0] wr_addr_lat; // Latched write address
37 wire addr_ok_w = (wr_addr_lat[3:2] < 4);
38 wire addr_ok_r;
39 wire [1:0] reg_idx_r;

```

```

40      // ---- WRITE PATH ----
41      always @(posedge aclk or negedge aresetn) begin
42          if (!aresetn) begin
43              awready <= 1'b0;
44              wready  <= 1'b0;
45              bvalid  <= 1'b0;
46              bresp   <= 2'b00;
47          end else begin
48              // AW: accept address
49              awready <= awvalid & ~awready & ~bvalid;
50              if (awvalid && awready) wr_addr_lat <= awaddr;
51              // W: accept data (simultaneously or after AW)
52              wready  <= wvalid & ~wready;
53              // Write to register when both channels accepted
54              if (wvalid && wready && awready) begin
55                  if (addr_ok_w) begin
56                      if (wstrb[0]) regs[wr_addr_lat[3:2]][ 7: 0] <= wdata[ 7: 0];
57                      if (wstrb[1]) regs[wr_addr_lat[3:2]][15: 8] <= wdata[15: 8];
58                      if (wstrb[2]) regs[wr_addr_lat[3:2]][23:16] <= wdata[23:16];
59                      if (wstrb[3]) regs[wr_addr_lat[3:2]][31:24] <= wdata[31:24];
60                      bresp <= 2'b00; // OKAY
61                  end else begin
62                      bresp <= 2'b10; // SLVERR: invalid address
63                  end
64                  bvalid <= 1'b1;
65              end
66              // B: complete write response
67              if (bvalid && bready) bvalid <= 1'b0;
68          end
69      end
70      // ---- READ PATH ----
71      assign addr_ok_r = (araddr[3:2] < 4);
72      assign reg_idx_r = araddr[3:2];
73      always @(posedge aclk or negedge aresetn) begin
74          if (!aresetn) begin
75              arready <= 1'b0;
76              rvalid  <= 1'b0;
77              rdata   <= {DATA_W{1'b0}};
78              rresp   <= 2'b00;
79          end else begin
80              arready <= arvalid & ~arready & ~rvalid;
81              if (arvalid && arready) begin
82                  rvalid <= 1'b1;
83                  if (addr_ok_r) begin
84                      rdata <= regs[reg_idx_r];
85                      rresp <= 2'b00; // OKAY
86                  end else begin
87                      rdata <= {DATA_W{1'b0}};
88                      rresp <= 2'b10; // SLVERR
89                  end
90              end
91              if (rvalid && rready) rvalid <= 1'b0;
92          end
93      end
94      // Initialize registers
95      integer k;
96      initial begin
97          for (k = 0; k < 4; k = k + 1) regs[k] = {DATA_W{1'b0}};
98      end
99  endmodule
100

```

RTL Design Explanation: AXI4-Lite has five independent channels:

1. **AW** (Write Address): Master sends address + control.
2. **W** (Write Data): Master sends data + byte strobes.
3. **B** (Write Response): Slave responds OKAY or error.
4. **AR** (Read Address): Master sends read address.
5. **R** (Read Data): Slave returns data + response.

The key protocol rule: **VALID must not wait for READY before being asserted**. The slave asserts **AWREADY/WREADY/ARREADY** as it can accept. The master must hold signals stable once **VALID** is asserted.

Common Interview Mistakes

- **Combinational VALID→READY loop:** If **AWREADY** combinational depends on **AWVALID**, and **AWVALID** depends on **AWREADY**, you have a deadlock. Always break the loop with a register.
- **AW and W channel race:** In AXI4-Lite, **W** data can arrive before the **AW** address. Latch both and process when both are received.
- **Missing BRESP:** Not sending **B**-channel response leaves the master waiting indefinitely (bus deadlock).

Interviewer's Perspective

Why asked: AXI4 is the industry-standard IP interconnect. Every SoC verification/design engineer must know this protocol cold.

Evaluating: 5-channel understanding, **VALID/READY** handshake, ordering rules, error response, byte strobe.

Impress: Reference the ARM AXI4-Lite spec (IHI0022). Mention the difference between AXI4-Lite (no bursts) and full AXI4 (burst up to 256 beats).

Follow-Up Questions

1. Extend this to a full AXI4 slave with burst support (**ARLEN**, **ARBURST**).
2. Add read-modify-write (**RMW**) atomicity for **W1C** registers.
3. How do you connect multiple AXI4-Lite slaves to a single master (interconnect)?
4. What is **WRAP** burst mode in AXI4 and when is it used?
5. How does AXI4 support out-of-order transactions (using **AXI ID**)?

Question 50

Design a Non-Blocking Cache with MSHR

EXPERT

Companies That Ask This Question

NVIDIA, AMD, Intel, Apple, ARM, Qualcomm

Problem Statement

Design the RTL logic for a simplified non-blocking cache controller with 2 MSHR (Miss Status Holding Register) entries. The cache should: (1) handle a load hit (1-cycle response), (2) handle a load miss (allocate MSHR, send fill request, replay when data returns), (3) detect a secondary miss to an already-pending MSHR entry (merge, no second fill). This is the type of question asked in CPU microarchitecture team interviews.

Verilog Solution - nb_cache_ctrl.v

```

1      // -----
2      // Module   : nb_cache_ctrl
3      // Purpose  : Non-Blocking Cache Controller with 2-entry MSHR
4      // Simplified: Direct-mapped cache, word-addressed
5      // -----
6      module nb_cache_ctrl #(
7          parameter CACHE_LINES = 16,
8          parameter DATA_W      = 32,
9          parameter ADDR_W       = 8,
10         parameter MSHR_ENTRIES = 2
11     )(
12         input  wire      clk,
13         input  wire      rst,
14         // CPU request interface
15         input  wire      cpu_valid,
16         input  wire [ADDR_W-1:0] cpu_addr,
17         output reg [DATA_W-1:0] cpu_rdata,
18         output reg        cpu_hit,    // Hit this cycle
19         output reg        cpu_stall,  // CPU must wait
20         // Memory fill interface (returns data from lower level)
21         input  wire      fill_valid,
22         input  wire [ADDR_W-1:0] fill_addr,
23         input  wire [DATA_W-1:0] fill_data,
24         // Memory request interface (send miss to lower level)
25         output reg        mem_req,
26         output reg [ADDR_W-1:0] mem_addr
27     );
28     localparam TAG_W = ADDR_W - $clog2(CACHE_LINES);
29     // Cache storage (direct-mapped)
30     reg [DATA_W-1:0] cache_data [0:CACHE_LINES-1];
31     reg [TAG_W-1:0]  cache_tag  [0:CACHE_LINES-1];
32     reg             cache_valid [0:CACHE_LINES-1];
33     // MSHR entries
34     reg [ADDR_W-1:0] mshr_addr  [0:MSHR_ENTRIES-1];
35     reg             mshr_valid [0:MSHR_ENTRIES-1];
36     wire [$clog2(CACHE_LINES)-1:0] idx = cpu_addr[$clog2(CACHE_LINES)-1:0];
37     wire [TAG_W-1:0] tag = cpu_addr[ADDR_W-1:$clog2(CACHE_LINES)];
38     wire cache_hit = cache_valid[idx] && (cache_tag[idx] == tag);

```



```

39    // MSHR hit: is this address already pending?
40    wire [MSHR_ENTRIES-1:0] mshr_match;
41    wire                      mshr_hit;
42    wire [MSHR_ENTRIES-1:0] mshr_free_mask;
43    integer i;
44    genvar g;
45    generate
46    for (g = 0; g < MSHR_ENTRIES; g = g + 1) begin : mshr_chk
47    assign mshr_match[g]    = mshr_valid[g] &&
48    (mshr_addr[g] == cpu_addr);
49    assign mshr_free_mask[g] = ~mshr_valid[g];
50    end
51    endgenerate
52    assign mshr_hit = |mshr_match;
53    wire mshr_full = &(mshr_valid[0 +: MSHR_ENTRIES]);
54    // Find free MSHR entry (priority encode)
55    wire mshr_free_idx = mshr_free_mask[0] ? 1'b0 : 1'b1;
56    always @(posedge clk) begin
57    if (rst) begin
58    for (i = 0; i < CACHE_LINES; i = i + 1)
59    cache_valid[i] <= 1'b0;
60    for (i = 0; i < MSHR_ENTRIES; i = i + 1)
61    mshr_valid[i] <= 1'b0;
62    cpu_hit <= 1'b0;
63    cpu_stall <= 1'b0;
64    mem_req <= 1'b0;
65    end else begin
66    mem_req <= 1'b0;
67    cpu_hit <= 1'b0;
68    cpu_stall <= 1'b0;
69    // --- Handle fill (data returning from memory) ---
70    for (i = 0; i < MSHR_ENTRIES; i = i + 1) begin
71    if (fill_valid && mshr_valid[i] &&
72    mshr_addr[i] == fill_addr) begin
73    // Write fill data into cache
74    cache_data[fill_addr[$clog2(CACHE_LINES)-1:0]] <= fill_data;
75    cache_tag [fill_addr[$clog2(CACHE_LINES)-1:0]] <=
76    fill_addr[ADDR_W-1:$clog2(CACHE_LINES)];
77    cache_valid[fill_addr[$clog2(CACHE_LINES)-1:0]] <= 1'b1;
78    mshr_valid[i] <= 1'b0; // Free MSHR entry
79    end
80    end
81    // --- Handle CPU request ---
82    if (cpu_valid) begin
83    if (cache_hit) begin
84    // HIT: return data immediately
85    cpu_rdata <= cache_data[idx];
86    cpu_hit <= 1'b1;
87    end else if (mshr_hit) begin
88
89    cpu_stall <= 1'b1;
90    end else if (!mshr_full) begin
91    // PRIMARY MISS: allocate MSHR, send memory request
92    mshr_addr[mshr_free_idx] <= cpu_addr;
93    mshr_valid[mshr_free_idx] <= 1'b1;
94    mem_req <= 1'b1;
95    mem_addr <= cpu_addr;
96    cpu_stall <= 1'b1;
97    end else begin
98
99    cpu_stall <= 1'b1;
100    end

```

```

101     end
102     end
103     end
104     endmodule
105

```

RTL Design Explanation:

- **MSHR (Miss Status Holding Register):** A table that tracks outstanding cache misses. Each entry stores the miss address. When the fill returns from memory, the MSHR is looked up to find which cache line to fill and which CPU requests to replay.
- **Primary miss:** First access to a missing line — allocates MSHR, issues fill request, stalls CPU.
- **Secondary miss:** Subsequent access to the same pending line — merges into the existing MSHR entry (no second fill request), stalls until the first fill returns.
- **Hit:** Cache tag matches — return data in one cycle, no stall.

Why Non-Blocking Caches Matter

A **blocking cache** stalls the entire CPU on every cache miss, waiting for memory. A **non-blocking cache** (also called *lookup-free cache*) allows the CPU to continue executing out-of-order instructions while the miss is being filled. Modern CPUs (Intel, AMD, ARM) use 10–20+ MSHR entries to hide memory latency. NVIDIA GPUs use warp scheduling (thousands of threads) to hide latency without MSHR.

Common Interview Mistakes

- **Not handling secondary misses:** Sending duplicate fill requests for the same address causes memory bandwidth waste and coherency issues.
- **Forgetting the fill-to-cache path:** The MSHR must also trigger the data write into the cache array when fill data returns.
- **MSHR full stall:** When all MSHR entries are occupied, the cache must stall and *not* accept new misses. This is called a **structural hazard**.

Pro Tips

- Real CPUs have complex MSHR designs: subentries for each beat of a cache line fill (e.g., 16 beats for a 64B cache line with 4B per transfer), as well as byte-granular store merging (write combining).
- MSHR count is a key microarchitectural parameter: fewer MSHR entries = more stalls under high miss rates. AMD Zen4 has 24 MSHR entries per L1D cache.
- The critical path for cache controller logic is often the tag comparison, MSHR CAM lookup, and cache write path — all in the same cycle.

Interviewer's Perspective

Why asked: This is the apex question for CPU microarchitecture positions. Only candidates who have deeply studied computer architecture can answer it correctly.

Evaluating: MSHR concept, primary/secondary miss handling, fill path, stall generation, structural hazard awareness.

Impress: Reference Kroft's original 1981 MSHR paper. Mention that virtually every modern high-performance processor uses non-blocking caches. Discuss how MSHR interacts with store buffers and cache coherence protocols (MESI).

Follow-Up Questions

1. Extend the MSHR to handle stores (write misses) with write combining.
2. How does cache coherence (MESI protocol) affect MSHR design?
3. What is a critical word first fill, and how does it reduce miss penalty?
4. Implement the fill replay mechanism: re-issue CPU request after fill.
5. How many MSHR entries does a modern out-of-order CPU need to saturate DRAM bandwidth at 64 ns latency and 64 GB/s bandwidth?

Practice Challenge

Extend this design to a 2-way set-associative cache with LRU replacement. Each set has 2 ways; the MSHR must track which way will be replaced. Add the LRU bit per set and update it on every cache access.

Verilog Quick Reference

How to Use This Reference

This appendix provides a condensed cheat sheet of the most important Verilog constructs, idioms, and rules you need to know for RTL interviews. Photocopy this page and review it the morning of your interview.

A.1 Blocking vs Non-Blocking Assignment

Blocking (=)	Non-Blocking (<=)
Sequential execution	Concurrent RHS evaluation
Use in combinational always	Use in sequential always @(posedge clk)
Models combinational logic	Models flip-flop behavior
Immediate update	Update at end of time step
Golden Rule: All assignments in a clocked always block MUST use <=	

A.2 Sensitivity List Rules

Sensitivity List Examples

```

1
2     always @(*) begin           // Verilog-2001
3         y = a & b;
4     end
5     always_comb begin          // SystemVerilog preferred
6         y = a & b;
7     end
8
9     always @(posedge clk) begin
10        if (rst) q <= 0;
11        else    q <= d;
12    end
13
14    always @(posedge clk or negedge rst_n) begin
15        if (!rst_n) q <= 0;
16        else        q <= d;
17    end
18

```

A.3 Latch Inference — How to Avoid

Latch Inference Rules

A latch is inferred in a combinational `always` block when:

- A signal is not assigned in all branches of an `if-else`.
- A `case` statement is not complete AND there is no `default`.
- An output is conditionally driven (“else” branch missing).

Fix: Always assign a default value at the top of the block: `y = 0;` before the `if/case`.

A.4 Key Verilog Operators

Operator	Description	Example
<code>^</code>	Bitwise XOR	<code>a ^ b</code>
<code>^</code> (unary)	XOR reduction (parity)	<code>^data</code>
<code>&</code> (unary)	AND reduction (all-ones)	<code>&data</code>
<code> </code> (unary)	OR reduction (any-set)	<code> data</code>
<code>>></code>	Arithmetic shift right	<code>\$signed(a) >> 2</code>
<code><< / >></code>	Logical shifts	<code>a << 3</code>
<code>[i +: n]</code>	Indexed part-select	<code>bus[i*8 +: 8]</code>
<code>[i -: n]</code>	Indexed part-select (down)	<code>bus[i*8+7 -: 8]</code>
<code>{}</code>	Concatenation	<code>{a, b, c}</code>
<code>{n{x}}</code>	Replication	<code>{8{1'b0}}</code>
<code>\$clog2(N)</code>	Ceiling log2	<code>\$clog2(256) = 8</code>
<code>\$signed()</code>	Signed cast	<code>\$signed(8'hFF) = -1</code>

A.5 Common Design Patterns

Must-Know RTL Patterns

```

1  // Pattern 1: Edge Detector
2  reg sig_d;
3  always @(posedge clk) sig_d <= sig;
4  assign pos_edge = sig & ~sig_d;
5  assign neg_edge = ~sig & sig_d;
6  // Pattern 2: Clock enable (preferred over clock gating in RTL)
7  always @(posedge clk)
8  if (en) q <= d; // Tool inserts ICG automatically
9  // Pattern 3: Parameterized counter with wrap
10 always @(posedge clk)
11 if (count == MAX-1) count <= 0;
12 else count <= count + 1;
13 // Pattern 4: Shift register
14 always @(posedge clk)
15 shift <= {shift[N-2:0], serial_in}; // Left shift, LSB in

```

```

16 // Pattern 5: Binary to Gray
17 assign gray = bin ^ (bin >> 1);
18 // Pattern 6: One-hot to binary (4-bit)
19 assign binary = {onehot[3:2], {onehot[3],onehot[1]}};
20 // Pattern 7: Majority vote (3-bit)
21 assign vote = (a & b) | (b & c) | (a & c);
22

```

A.6 FIFO Pointer Full/Empty Rules

Condition	Rule
Empty	<code>wr_ptr == rd_ptr</code> (all bits equal)
Full	MSBs differ, lower bits equal (extra bit method)
Count	<code>wr_ptr - rd_ptr</code> (unsigned, power-of-2 depth)
Async	Convert to Gray code before synchronizing pointers

A.7 FSM Design Rules

Interview-Ready FSM Checklist

1. Always use 3-block style: state register, next-state logic, output logic.
2. Declare states with `typedef enum` (SystemVerilog) or `localparams`.
3. Include a `default` state in the next-state `case`.
4. Moore output depends only on state; Mealy output depends on state AND inputs.
5. Draw the state diagram before writing code.
6. For safety-critical designs, register the outputs (Moore registered).
7. One-hot encoding: N states use N flip-flops; fast but area-expensive.
8. Binary encoding: $\lceil \log_2 N \rceil$ flip-flops; compact but slower.

A.8 Clock Domain Crossing Cheat Sheet

Signal Type	CDC Method
Single-bit control	2-flop synchronizer
Single-bit pulse (fast→slow)	Toggle synchronizer + edge detect
Multi-bit bus (rare transitions)	Gray encoding + 2-flop sync
Multi-bit data stream	Async FIFO (Gray-coded pointers)
Low-rate data (any width)	4-phase handshake

Interview Preparation Strategy

B.1 30-Day Study Plan

Week	Focus	Questions / Topics
Week 1	Beginner Mastery	Q1–Q10, practice each on paper, simulate all in Icarus Verilog
Week 2	Intermediate I	Q11–Q20, focus on FIFO, UART, SPI, CDC, FSMs
Week 3	Intermediate II + Advanced I	Q21–Q35, timers, arbiters, APB, AXI streams
Week 4	Advanced II + Expert	Q36–Q50, CLA, OoO queue, AXI4-Lite, cache controller
Daily Practice: 1 question coded by hand, 1 simulated, 1 reviewed		

B.2 On-Site Interview Tips

How to Excel in RTL Interviews

1. **Think aloud.** Say what you are thinking. Interviewers evaluate your reasoning, not just your final answer.
2. **Draw first.** Sketch a block diagram or state diagram before writing Verilog. This shows architectural thinking.
3. **State your assumptions.** Before coding, say: “I’ll use synchronous reset, ANSI port style, and parameterize the width.”
4. **Name your signals clearly.** `wr_ptr_gray` is better than `wp2`.
5. **Mention edge cases.** Bring up simultaneous read/write, all-zeros LFSR lockup, metastability — before the interviewer asks.
6. **Know your timing.** Be ready to discuss setup/hold, critical paths, and what limits clock frequency.
7. **Reference real tools.** Mention ModelSim, VCS, Synopsys DC, Cadence Genus, Vivado — shows industry familiarity.

B.3 Topics by Company Focus

Company	Strong Interview Focus
NVIDIA	GPU microarchitecture, pipeline, CDC, async FIFO, warp scheduling
AMD	CPU pipeline (Zen), cache hierarchy, prefetching, OoO execution
Intel	SoC integration, PCIe, DDR controller, power management (RAPL)
Qualcomm	Mobile SoC, DSP, modem baseband, low-power clock gating
Apple	Custom CPU (M-series), AXI, neural engine, performance optimization
ARM	AMBA protocols (AXI, AHB, APB), processor IP, TrustZone
Broadcom	Networking ASIC, parser, scheduler, packet buffer, CDR
Synopsys	Synthesis algorithms, timing, EDA tool RTL expectations
Cadence	Verification methodology (UVM), functional coverage, FPGA prototyping
Samsung	Memory controller, LPDDR, NAND flash interface, display IP
TI	Mixed-signal control, PWM, timer, ADC/DAC interface
Marvell	Storage controller, Ethernet MAC/PHY, data path pipelining
MediaTek	Display pipeline, multimedia SoC, ISP, AI accelerator

Recommended Reading

Essential RTL Engineering Library

Every serious RTL engineer should own (or have read) the following books.

1. **Samir Palnitkar** — *Verilog HDL: A Guide to Digital Design and Synthesis* (2nd ed.) — The definitive Verilog reference book.
2. **Michael Ciletti** — *Advanced Digital Design with Verilog HDL* — Deep RTL design with synthesis examples.
3. **Chris Spear & Greg Tumbush** — *SystemVerilog for Verification* — Essential for verification engineers; useful for RTL engineers too.
4. **Neil Weste & David Harris** — *CMOS VLSI Design: A Circuits and Systems Perspective* — Physical design intuition for RTL engineers.
5. **John Hennessy & David Patterson** — *Computer Organization and Design and Computer Architecture: A Quantitative Approach* — The bible of computer architecture. Essential for microarchitecture interviews.
6. **Clifford Cummings** — SNUG (Synopsys Users Group) Papers — Free online. Essential CDC, FSM, and synthesis papers. Particularly: “Simulation and Synthesis Techniques for Asynchronous FIFO Design” (2002).
7. **Don Mills & Clifford Cummings** — “RTL Coding Styles That Prevent Simulation/Synthesis Mismatches” (SNUG 1999) — Prevents common mistakes.
8. **ARM IHI0022** — *AMBA AXI and ACE Protocol Specification* — Free download from ARM. Every SoC engineer must study this.
9. **IEEE Std 1364-2001** — *IEEE Standard for Verilog Hardware Description Language* — The official standard.
10. **IEEE Std 1800-2017** — *IEEE Standard for SystemVerilog* — The official SystemVerilog standard.

50 Most Asked Verilog Coding Interview Questions

A Practical Guide to RTL Coding

Interviews

Kittu Patel

50 Real Interview Questions • Production Verilog Code
Beginner to Expert • Industry Tips & Tricks

NVIDIA • AMD •

Intel • Qualcomm • Apple • ARM • Broadcom

First Edition • 2025